AI Infra M2 分布式训练学习笔记：从显存账本到 5D 并行

数据来源：本笔记的章节骨架、术语体系、显存账本口径、通信量公式与绝大部分数字，取自 AIInfraGuide「模块三：分布式训练」的 17 篇公开页面（第 1～11 章，共 17 个 URL），访问日期 **2026-11**。其中 6 篇是长篇正文（1.1 分布式训练总论、1.2 环境搭建与分布式启动、2.1 集合通信原语详解、3.1 优化器、4.1 数据并行详解、4.2 PyTorch 数据并行从原理到实战），其余 11 个页面是章节扉页（含完整章节结构、公式与选型表）。 页面前缀：`https://caomaolufei.github.io/AIInfraGuide/distributed/模块三-分布式训练/`。

补充来源（用于交叉核对机制、补齐原文未展开的部分，文中标注「补充」）：Rajbhandari et al. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*（SC 2020）、Shoeybi et al. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*（2019）、Huang et al. *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism*（NeurIPS 2019）、Narayanan et al. *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*（SC 2021，1F1B 与 Interleaved 调度）、Korthikanti et al. *Reducing Activation Recomputation in Large Transformer Models*（2022，序列并行与激活量公式）、PyTorch 官方文档（DDP / FSDP / `torch.distributed` / torchrun）、NVIDIA NCCL 官方文档与 nccl-tests、DeepSpeed 官方文档（ZeRO / ZeRO-Offload / ZeRO-Infinity / Ulysses）、Dao et al. *FlashAttention* 系列、Liu et al. *Ring Attention with Blockwise Transformers for Near-Infinite Context*（2023）、Jacobs et al. *DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models*（2023）、DeepSeek-AI *DeepSeek-V3 Technical Report*（2024，aux-loss-free 负载均衡）。

标注约定：`【补充】` = AIInfraGuide 原文没有、由上述权威来源或通用知识补齐；`【推断】` = 基于材料做的推理性结论，未经原文或文档直接陈述；`【量级】` = 只保证数量级正确，不对具体尾数负责。

版权声明：AIInfraGuide 原文版权归原作者所有，本笔记为个人学习用途的二次整理、重组与批注，不替代原文；引用原文观点与数据处尽量保留其表述与限定条件。论文、PyTorch / NVIDIA / DeepSpeed 文档版权归各自权利人所有。若本文与原文或官方文档冲突，以原文与官方文档为准。

一个必须先说清的口径问题：本模块所有「16Ψ / 12Ψ / 2Ψ / 3Ψ」类公式，默认前提都是 **BF16（或 FP16）混合精度 + Adam/AdamW**。换优化器（SGD、Lion、Adafactor）、换精度（FP32 全量、FP8）、换框架（DeepSpeed 的 `zero_optimization` 配置项、PyTorch 的 FSDP 策略）都会让系数变化。**记住推导方法，不要背死系数**——这是本模块唯一真正可迁移的能力。

1.1 两个正交的问题，别混在一起谈

1.1.1 「装不下」到底差多少

1.1.2 「跑不完」到底慢多少

1.1.3 一个贯穿全模块的判断

1.2 训练显存账本：手算一遍就懂

1.2.1 静态显存四件套（16Ψ 的标准口径）

1.2.2 16Ψ 与 18Ψ：口径差异从哪来

1.2.3 激活值：真正的显存刺客

1.2.4 显存估算速查表
1.3 五大并行策略全景
1.3.1 五策略的定量对比（同一把尺子）
1.4 通信–显存–计算三角
1.5 选型：带宽决定作用域
1.5.1 决策树
1.6 本章小结与自检
2.1 为什么单独讲通信
2.2 点对点：一切的基础
2.3 七个集合原语：输入 → 输出
2.3.1 Broadcast：一份数据复制给所有人
2.3.2 Scatter / Gather：分发与收集
2.3.3 Reduce：规约到一个 rank
2.3.4 AllReduce：规约结果人人有份（核心）
2.3.5 ReduceScatter：规约后只留自己那份
2.3.6 AllGather：分片拼成完整数据
2.3.7 All–to–All：人人给人人发不同数据
2.3.8 贯穿全模块的恒等式
2.4 Ring AllReduce：为什么通信量与卡数无关
2.4.1 朴素实现的带宽瓶颈
2.4.2 Ring 的两阶段
2.4.3 通信量推导（必会）
2.4.4 带宽最优性与拓扑选择
2.5 通信量的量化方法
2.5.1 统一记号
2.5.2 三类原语的每卡通信量（Ring 实现，大 N 近似）
2.5.3 通信时间：α–β 模型
2.5.4 计算/通信重叠
2.6 NCCL：环境变量与调优
2.6.1 诊断类
2.6.2 干预类（指定路径）
2.6.3 二分排障开关（用完必须关）
2.6.4 性能调优类
2.6.5 一个可复现的验证实验
2.7 本章小结与自检
3.1 优化器在训练循环里的位置
3.2 演进主线：每加一样东西，多存一份状态
3.3 显存账：12 字节/参数
3.3.1 为什么优化器状态必须 FP32
3.4 大 Batch 优化器：LARS 与 LAMB
3.5 动手：把优化器的账本翻出来看
3.6 本章小结与自检
4.1 数据并行解决的是「太慢」
4.2 数学基础：为什么必须「取平均」而不是「求和」
4.3 第一代 DataParallel：单进程多线程的三个致命缺陷
4.4 第二代 DDP：每卡一进程 + 去中心化 AllReduce
4.4.1 Bucket 机制：把通信藏进计算
4.4.2 DDP 的四个常用参数
4.4.3 DDP 的扩展性天花板
4.5 DDP 的根本局限与通信账
4.5.1 每步 2Ψ 的出处（复习第 2 章）
4.6 一个正确的 DDP 训练脚本（骨架）
4.6.1 五个必须记住的细节
4.7 本章小结与自检
5.1 冗余在哪里
5.2 ZeRO–1：切优化器状态
5.3 ZeRO–2：连梯度也切
5.4 ZeRO–3：连参数也切
5.5 FSDP：PyTorch 的原生实现
5.5.1 FSDP 训练代码
5.5.2 三个必须会的 FSDP 细节
5.5.3 FSDP2：`fully_shard`
5.6 ZeRO–Offload 与 ZeRO–Infinity
5.7 选型：一张表定方案
5.8 本章小结与自检
6.1 为什么 DP/FSDP 到头了
6.2 Column Parallel Linear：按列切
6.3 Row Parallel Linear：按行切
6.4 Transformer Block 的 TP 切分：通信插入位置推导
6.4.1 通信量
6.5 序列并行 SP：补上 TP 的漏洞
6.6 GQA / MQA 下的 TP：KV Head 不够切怎么办
6.7 手写一个最小 TP（Column + Row）

6.8 本章小结与自检
7.1 基本思想与第一个代价
7.2 三代调度：同一个 Bubble 公式，不同的显存
7.2.1 GPipe：切 micro-batch
7.2.2 1F1B：省显存的关键一步
7.2.3 Interleaved 1F1B：虚拟 Stage
7.3 四种调度横向对比
7.3.1 Bubble 的绝对时间
7.4 PP 的四个工程挑战
7.5 手写一个 PP 调度模拟器（理解 Bubble 的最快方式）
7.6 本章小结与自检
8.1 四种数值格式
8.1.1 BF16 混合精度的完整流程
8.1.2 FP16 vs BF16：为什么大模型选 BF16
8.1.3 Loss Scaling：FP16 的必要补丁
8.1.4 FP8 训练：能省但要小心
8.2 梯度累积：小显存模拟大 batch
8.3 Activation Checkpointing：用计算换显存
8.4 显存优化全景：一张表收束
8.5 本章小结与自检
9.1 序列长度墙：为什么别的并行都不管用
9.2 FlashAttention：CP 的前置知识
9.3 Ring Attention：环传 KV
9.4 DeepSpeed-Ulysses：Head → 序列的 All-to-All 换轴
9.5 Megatron Context Parallel
9.6 三种方案对比
9.7 一个最小 Ring Attention 骨架
9.8 本章小结与自检
10.1 稠密 FFN → 稀疏 MoE
10.1.1 Router 与容量因子
10.2 Expert Parallelism
10.3 两次 All-to-All：MoE 的通信主场
10.4 负载均衡：MoE 的头号工程问题
10.5 EP × DP × TP × PP 的组合
10.6 本章小结与自检
11.1 为什么必须组合
11.2 通信域映射：本章最重要的一节
11.3 rank 编排：从一维编号到多维坐标
11.4 拓扑设计实战：64 卡（8 节点 × 8 GPU）
11.5 显存与吞吐的联合估算
11.6 一张「选型决策表」收尾
11.7 本章小结与自检
12.1 三种规模与带宽分层
12.2 一卡一进程与三个编号
12.3 torchrun：一条命令拉起一堆进程
12.3.1 注入的六个环境变量
12.3.2 单机多卡
12.3.3 多机：c10d 弹性会合
12.4 子通信组：为 3D 并行铺路
12.5 排障流程
12.6 首次跑多卡的五个坑
12.7 本章小结
14.1 分章复习清单
14.2 自测题（20 题，答案在后）
14.3 答案
15.1 一手来源（AIInfraGuide 模块三，访问日期 2026-11）
15.2 补充来源（论文与官方文档）
15.3 阅读顺序建议

第 1 章 分布式训练总论：装不下、跑不完与显存账本

> 对应 AIInfraGuide 模块三 第 1 章（`第1章-分布式训练总论` 扉页 + `11-分布式训练总论` 正文）。

1.1 两个正交的问题，别混在一起谈

几乎所有分布式训练的入门困惑，都来自把两件事混为一谈：

```
┌──────────────────────────────────────────────────────────────┐
│  问题 A：装不下（空间）          问题 B：跑不完（时间）          │
│  ──────────────────────          ──────────────────────      │
│  单卡显存 < 训练状态              单卡算力 × 可接受时间 < 总计算量 │
│                                                              │
│  解法：切分（split）              解法：复制并行（replicate）     │
│  ├─ ZeRO / FSDP 切状态            └─ 数据并行 DP / DDP         │
│  ├─ TP 切单层矩阵                                            │
│  ├─ PP 切层                       两者可以叠加，且叠加后互相制约  │
│  └─ SP / CP 切序列                                           │
└──────────────────────────────────────────────────────────────┘
```

原文的比喻很到位：单卡训大模型，像让一个人用一张小书桌整理一整座图书馆——桌子太小摊不开（显存），一个人搬一辈子也搬不完（算力）。两个困境的解法方向完全不同。

1.1.1 「装不下」到底差多少

先建立一个粗略但极有用的量级结论：**BF16 混合精度 + Adam 下，每个参数约需 16 字节静态显存**（下一节详细拆）。用 LLaMA 系列估一下（只算静态，不含激活值）：

模型	参数量 Ψ	静态显存 ≈ 16Ψ	单卡 H100 80GB 能否装下
LLaMA 7B	7×10^9	≈ 112 GB	❌ 装不下
LLaMA 70B	70×10^9	≈ 1120 GB	❌ 需 14 张 80GB
LLaMA 405B	405×10^9	≈ 6480 GB	❌ 需数十卡

注意这张表最反直觉的一行：连「小号」的 7B，光静态显存就 112 GB，已经超过一张 H100 的 80 GB——而且还没算前向堆积的激活值。所以「7B 能不能单卡训」的正确答案是「不能」，尽管它的推理（BF16 权重 14 GB）在单卡上轻轻松松。

1.1.2 「跑不完」到底慢多少

大模型训练的计算量有个广为引用的经验公式【原文】：

```
C ≈ 6 · Ψ · D          C = 总浮点运算次数（FLOPs）
                       Ψ = 参数量
                       D = 训练 token 数
```

这个 `6` 的来源是：前向每个参数每个 token 约 2 次浮点运算（一次乘、一次加），反向约 4 次（对输入求导 2 次 + 对权重求导 2 次），合计 6。

代入 70B 模型训 1T（10^{12}）token：

```
C ≈ 6 × (70×10⁹) × 10¹² = 4.2×10²³ FLOPs
```

单张 H100 的 BF16 稠密算力峰值约 10^{15} FLOP/s，乐观假设 50% 的 MFU（Model FLOPs Utilization）：

```
T = 4.2×10²³ / (10¹⁵ × 0.5) ≈ 8.4×10⁸ 秒 ≈ 26.6 年
```

单卡要训二十多年。而 N 卡数据并行理论上能把时间压到接近 1/N，1024 卡可降到 **约 9 天量级**【原文】。

> 【补充】这里有两个必须自己心里有数的修正项：(1) `6ΨD` 只算矩阵乘，不含 Attention 的 `O(s²)` 项，短序列下可忽略、长序列（32K+）下不可忽略；(2) 50% MFU 对 2023 年后的 H100 集群是偏乐观的工程值，实际端到端 MFU 常在 35%~50% 区间【量级】。两者一叠加，「26.6 年」和「9 天」都应该当量级看，而不是当预测。

1.1.3 一个贯穿全模块的判断

把 A、B 两个问题分开后，后面每一章的位置就清楚了：

章节	在解决 A（装不下）还是 B（跑不完）	手段
第 2 章 集合通信	都不是，是**两者的公共成本**	把通信本身做快
第 3 章 优化器	A（算清账）+ B（大 batch 稳定性）	搞清 12Ψ 从哪来
第 4 章 DP / DDP	B	复制模型、切数据
第 5 章 ZeRO / FSDP	A	切分训练状态
第 6 章 TP / SP	A（单层太大）	切层内矩阵
第 7 章 PP	A（层数太多）	切层
第 8 章 混合精度 / 重计算	A	降精度、时间换空间
第 9 章 CP	A（序列太长）	切序列
第 10 章 MoE	A + B（稀疏扩容）	切专家
第 11 章 3D 并行	A + B（组合）	拓扑设计

1.2 训练显存账本：手算一遍就懂

这是全模块最重要的基本功。训练显存分两块：**静态显存**（与训练步无关、常驻）和**激活值显存**（随 batch / 序列长度变化）。

1.2.1 静态显存四件套（16Ψ 的标准口径）

以最主流的 **BF16 混合精度 + Adam** 为例，设参数量为 Ψ，逐项列账（单位 Bytes）：

组成	精度	每参数字节	作用	生存期
模型参数（工作副本）	BF16	2Ψ	前向/反向实际参与计算的权重	全程
梯度	BF16	2Ψ	反向产出，与参数同形状	反向→更新
优化器：FP32 主权重	FP32	4Ψ	高精度参数，避免更新量被舍入吃掉	全程
优化器：一阶动量 m	FP32	4Ψ	梯度的指数滑动平均（Adam）	全程
优化器：二阶动量 v	FP32	4Ψ	梯度平方的指数滑动平均（Adam）	全程
合计		**16Ψ**		

用公式写出来：

```
M_static = 2Ψ + 2Ψ + 12Ψ = 16Ψ  Bytes
           └参数┘ └梯度┘ └── Adam 状态（4+4+4）──┘
```

代入 $\Psi = 7\times10^9$：

```
参数     2 × 7B = 14 GB
梯度     2 × 7B = 14 GB
优化器 12 × 7B = 84 GB
──────────────────────
合计            = 112 GB
```

这张账本最重要的结论不是 112，而是 84/112 = 75%：优化器状态占了静态显存的四分之三，参数本身只占 12.5%。这直接解释了 ZeRO 的设计选择——第一刀就砍优化器状态，因为它的性价比最高（第 5 章）。

1.2.2 16Ψ 与 18Ψ：口径差异从哪来

原文明确提到：「很多资料写每参数 16~18 字节，那个浮动来自实现细节差异（比如是否额外保留一份 FP32 梯度、是否用 FP32 主梯度累加）」。这句话值得展开成一张表，因为面试和评审里最容易被追问的就是「你到底按几字节算」。

口径	每参数字节	拆法	隐含假设	典型出处
紧凑	16Ψ	2 + 2 + 12	只有一份 BF16 参数、一份 BF16 梯度、一份 FP32 主权重 + m + v	ZeRO 论文的经典 16Ψ、本模块原文
常见工程值	16~18Ψ	16Ψ + 0~2Ψ	额外一份 BF16 参数副本或梯度通信缓冲（bucket / AllGather 输出）【推断】	各类工程估算经验值
保守	20Ψ	2 + 2 + 4（FP32 梯度）+ 12	反向直接产出并保留 FP32 梯度（不做 BF16 压缩）	部分框架在 `fp32` 梯度累加下的行为【推断】

三条使用建议：

1. **做容量规划时用 18Ψ 或 20Ψ，做理论推导时用 16Ψ**。前者留余量，后者对齐论文。
2. 差额那 2Ψ 在不同框架里对应不同东西（通信 bucket、双缓冲、对齐 padding），**不要试图在公式层面统一**，标一句「实现相关」就够了。
3. 真正决定你 OOM 与否的往往不是这 2Ψ，而是**激活值 + 显存碎片**。原文给的经验是：估出模型状态后，给峰值再乘 **1.2~1.5** 的余量。

1.2.3 激活值：真正的显存刺客

激活值是前向为反向保留的中间结果。原文给的结构性结论是：对 Transformer，单层激活值量级正比于 `b · s · h`（b = batch，s = 序列长度，h = 隐藏维），全模型再乘层数 L。

【补充】Korthikanti et al. 在 *Reducing Activation Recomputation in Large Transformer Models* 中给出的每层激活量估算为 `s·b·h·(34 + 5·a·s/h)` 字节（a = 注意力头数，BF16、无重计算、无并行）。这个式子的两个信息比数字本身更重要：

- 常数项 `34` 是**与序列长度无关的线性项**（LayerNorm、QKV、FFN 中间激活等一堆 h 维张量的字节数之和）；
- `5·a·s/h` 是**与 s 成正比的二次项**，来自 Attention 的 score 矩阵 `s×s`。当 `s` 变大时，这一项会反超常数项。

也就是说，**短序列时激活随 s 线性涨，长序列时随 s^2 涨**。这解释了为什么 128K 上下文的训练必须上第 9 章的 CP，而不是「把 batch 调小一点」。

> 但别把这公式当圣旨：它的常数依赖具体实现（是否 fused LayerNorm、是否用 FlashAttention、FFN 扩展比是 4 还是 8/3）。**用法是量级估算 + 相对比较**（开 checkpointing 前后差几倍、开 SP 前后差几倍），不是精确预测。

1.2.4 显存估算速查表

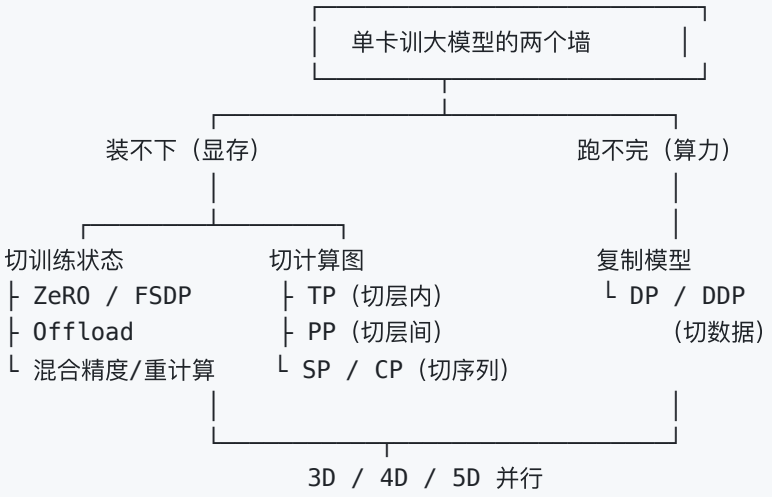
项目	公式（Bytes）	取决于	能否压缩
参数	2Ψ	参数量	可（降精度 / 切分）
梯度	2Ψ	参数量	可（切分 / 累积）
Adam 状态	12Ψ	参数量	可（切分 / 卸载 / 换优化器）
静态合计	**16Ψ**	参数量	
激活值	∝ b·s·h·L（长序列时含 s^2 项）	batch / 序列 / 宽深	可（checkpointing / SP / CP / 小 batch）
临时缓冲	通信 bucket、AllGather 输出、workspace	框架实现	部分可调

30 秒估算流程（后面速查文档会再给一遍）：

```
1）静态 = 16 × Ψ(单位 B)                          → 7B ≈ 112 GB
2）激活 = 按 1.2.3 量级估，或用（1）的 10%~30% 起猜【量级】
3）峰值 =（静态 + 激活）× 1.2~1.5（碎片与缓冲）
4）对比单卡容量（80GB / 141GB H200 / 192GB B200）
5）除以并行度，看你打算用哪种切分把峰值压到容量以下
```

1.3 五大并行策略全景

每种策略本质上都在回答两个问题：**切什么**（决定省不省显存）、**怎么通信**（决定快不快）。



策略	切什么	主要通信	解决什么	典型作用域
数据并行 DP / DDP	切数据，模型不切	梯度 AllReduce	跑不完（加速）	可跨机
ZeRO / FSDP	切优化器状态 / 梯度 / 参数	ReduceScatter + AllGather	装不下（省显存）	可跨机（HYBRID_SHARD 分层）
张量并行 TP	切单层的矩阵运算	层内 AllReduce	单层太大	必须机内 NVLink
流水线并行 PP	切网络层（按深度分段）	段间传激活值（P2P）	层数太多	可跨机
序列 / 上下文并行 SP / CP	沿序列维切	AllGather / All-to-All / P2P	序列太长	通常与 TP 同域
专家并行 EP	切 MoE 专家	All-to-All ×2	稀疏扩容	可跨机（带宽敏感）

逐条建立直觉：

- **DP / DDP**：每卡一份完整模型，各算各的数据分片，反向 AllReduce 同步梯度。**不省显存**（每卡仍装完整模型），但能近线性加速。核心约束是「每卡都装得下完整模型」。

- **ZeRO / FSDP**：DDP 的显存优化版。既然每卡都存了一份完全相同的 16Ψ，不如分片存储、用时再 AllGather 拼回来。它把「冗余」变成「按需」。
- **TP**：把一层的大矩阵按行/列切到多卡，每卡算一部分再合并。通信频繁且量大（每层 2 次 AllReduce），**只适合机内 NVLink**。
- **PP**：把模型按层切成几段，激活值像流水线一样在段间传递。通信量小（只在段边界传一次 b×s×h），可跨机，代价是 Bubble。
- **SP / CP**：沿序列长度维切分。SP 切的是 TP 未覆盖的 LayerNorm/Dropout 区域（省激活），CP 切的是 Attention 计算本身（省 s^2）。

1.3.1 五策略的定量对比（同一把尺子）

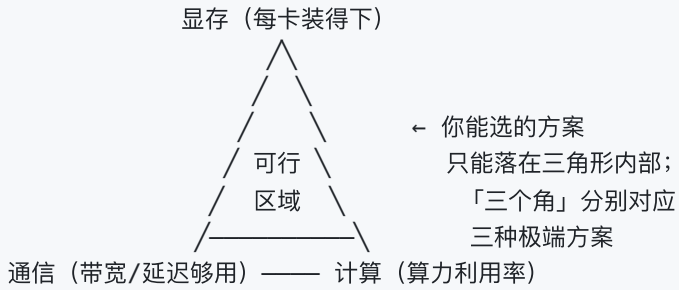
以 7B 模型、8 卡、BF16 + Adam 为例（只算静态显存，N=8）：

策略	每卡静态显存	每步每卡通信量	相对 DDP 的吞吐	主要代价
DDP	16Ψ ≈ 112 GB	2Ψ	1.0（基准）	装不下 7B
ZeRO-1	4Ψ + 12Ψ/8 ≈ 38.5 GB	2Ψ + AllGather	略低于 DDP	通信略增
ZeRO-2	2Ψ + 14Ψ/8 ≈ 26 GB	≈ 2Ψ（RS+AG）	略低于 DDP	通信略增
ZeRO-3 / FSDP FULL_SHARD	16Ψ/8 = 14 GB	3Ψ	明显低于 DDP	通信 +50%
TP=8	16Ψ/8 = 14 GB	每层 2 次 AllReduce	受通信限制明显	必须机内
PP=8	≈ 16Ψ/8 + 激活	段间 P2P（很小）	Bubble 损失	需要 M 个 micro-batch

> 这张表里「相对吞吐」一列只能定性给方向，**绝对值强依赖网络**：8 卡 NVLink 全互联时 ZeRO-3 的额外通信基本能被重叠掉；跨机 PCIe 时可能掉一半以上【推断】。

1.4 通信–显存–计算三角

这是本模块的「第一性原理」。任何并行方案都在这三个约束之间做取舍，而且**三者不可能同时最优**。



靠近「显存」角：FSDP FULL_SHARD / ZeRO-3 / Offload → 省显存但费通信
靠近「通信」角：DDP → 通信最省但显存最贵
靠近「计算」角：TP + PP 大并行度 → 算力利用率高但依赖高带宽互联

把三角落成三条可计算的判据：

判据	公式	什么时候会失败
装得下	`16Ψ/N_shard + 激活 + 缓冲 ≤ 卡容量`	参数/激活超容量 → OOM
通信盖得住	`T_comm ≤ T_comp`（重叠后）	通信暴露 → GPU 空转，扩展性崩塌
算得动	`T_total = T_comp + T_comm_exposed + T_bubble`	Bubble/等待占比高 → 加卡不提速

「通信盖得住」这条最容易被忽略，因为它是唯一随时间变化的：卡数少时通信能被计算掩盖，卡数多到一定程度通信就必然暴露。原文在 DDP 一节给了一个具体的工程观察：在 512+ GPU 规模下，DP 的通信开始受限于**环延迟（ring latency）**——信号绕 Ring 传播一圈的时间——通信无法再被完全重叠，此时应该换维度（TP/PP/ZeRO），而不是继续堆 DP。

1.5 选型：带宽决定作用域

选型的第一原则只有一句话：**通信带宽决定策略的作用域**。

互联	带宽量级	适配策略	为什么
NVLink / NVSwitch（机内）	数百 GB/s（如 NVLink 900 GB/s）	TP、SP、CP、EP	通信最频繁，必须机内
InfiniBand / RoCE（机间）	数百 Gbps（如 400 Gbps ≈ 50 GB/s）	PP、DP、ZeRO	通信稀疏，可跨机
以太网	更低（10~100 Gbps）	仅 DP 这类低频通信勉强可用	带宽不足

> 注意单位陷阱：**NVLink 用 GB/s，IB 用 Gbps**。400 Gbps = 50 GB/s，900 GB/s = 7200 Gbps。两者差 18 倍，不是一个数量级的问题——这就是「TP 出不了机」的物理原因。

1.5.1 决策树

```
开始：要训多大的模型？
│
├─ Q1 单卡装得下「参数+梯度+优化器+激活」？
│    ├─ 是 → DDP（简单高效，通信最省）
│    └─ 否 → Q2
│
├─ Q2 切分后单卡能装下？
│    ├─ 差一点 → FSDP SHARD_GRAD_OP / ZeRO-2（通信较省）
│    ├─ 差很多 → FSDP FULL_SHARD / ZeRO-3
│    ├─ 多机 → FSDP HYBRID_SHARD（机内分片 + 机间 DP）
│    └─ 还是不行 → Q3
│
├─ Q3 瓶颈在哪？
│    ├─ 单层太大（大 FFN / 大 Embedding / 大 vocab）→ 叠 TP（机内）
│    ├─ 层数太多 / 想跨机扩展 → 叠 PP
│    ├─ 序列太长（32K+）→ 叠 CP
│    └─ 想要稀疏扩容（降每 token 计算量）→ 换 MoE + EP
│
└─ Q4 组合成 3D/4D/5D：world_size = TP × PP × DP × EP × CP
```

原文给的两条态度性建议值得抄下来：

- ✅ **推荐路径**：从最简单的够用方案起步，不够了再加维度。单卡装得下就 DDP；装不下先上 FSDP/ZeRO；还不行再按瓶颈叠 TP/PP/CP；千亿级才需要全套 3D 并行。
- ❌ **不推荐**：一上来就堆 3D 并行。每多一个并行维度都会显著增加调试复杂度（多组 NCCL communicator、rank 编排、checkpoint 分片格式）和通信开销，过早优化得不偿失。

1.6 本章小结与自检

必须记住的六个数字：

1. `16Ψ` —— BF16 + Adam 的静态显存（2 参数 + 2 梯度 + 12 优化器）。
2. `75%` —— 优化器状态在静态显存里的占比（12/16）。
3. `112 GB` —— 7B 模型的静态显存，超过单张 80GB 卡。
4. `6ΨD` —— 训练总计算量的量级公式。
5. `900 GB/s vs 50 GB/s` —— NVLink 与 IB 的带宽比，决定 TP 出不了机。
6. `1.2~1.5` —— 容量规划的安全系数。

自检题（答案在第 14 章）：

1. 为什么 7B 模型「推理能单卡跑，训练不能单卡训」？分别算一遍。
2. 如果只用 SGD（无 momentum），16Ψ 会变成多少？
3. 一个 13B 模型用 BF16 + Adam，静态显存是多少？需要几张 80GB 卡才「刚好」装下（不考虑激活）？

4. 原文说「激活值是可调成本」，为什么？列出至少三种压缩激活的手段。

5. 为什么 TP 必须限制在机内，而 PP 可以跨机？用通信量公式解释。

第 2 章 集合通信原语：并行训练的字母表

> 对应 AIInfraGuide 模块三 第 2 章（`第2章-集合通信原语` 扉页 + `21-集合通信原语详解` 正文）。

2.1 为什么单独讲通信

原文给了一句可以当座右铭的话：**并行 = 切分 + 通信**。切分决定显存怎么省、负载怎么分；切分之后各卡手里只有「半成品」，必须靠通信拼成正确结果。

更准确地说：**切分决定显存，通信决定速度**。一个并行策略快不快，往往不取决于它怎么切，而取决于它每一步要传多少数据、走什么网络。

先给全景——「哪个策略用哪个原语」：

原语	谁在用它	传的数据
AllReduce	DDP 梯度同步、TP 输出合并	完整梯度 / 部分和
ReduceScatter	ZeRO-2/3、FSDP 反向	梯度（规约后分片）
AllGather	ZeRO-3 前向、FSDP 重组参数	参数分片
All-to-All	MoE 专家并行、Ulysses	token / 激活
Broadcast	初始化时同步参数	完整参数
Reduce	规约到根（少用）	完整数据
send / recv	PP 传激活值	激活值

2.2 点对点：一切的基础

`send(tensor, dst)` / `recv(tensor, src)` 是最朴素的两进程通信。集合通信再花哨，底层都是它们的组合。PP 的 Stage 之间传激活值靠的就是它。

这里有本模块第一个真正的 bug 陷阱——死锁：

```
# ❌ 危险写法：两个进程都先 send 再 recv
dist.send(tensor, dst=peer)   # rank0 等 rank1 来收
dist.recv(tensor, src=peer)   # rank1 等 rank0 来收 → 双方都卡在 send
```

如果两个进程都用阻塞式 send 先发后收，且底层缓冲区不足以暂存，双方就会都卡在 send 上等对方来 recv。

```
# ✅ 方案 A：错开顺序
if rank == 0:
    dist.send(tensor, dst=1)
    dist.recv(tensor, src=1)
else:
    dist.recv(tensor, src=0)
    dist.send(tensor, dst=0)
```

```
# ✅ 方案 B：异步原语，先发起再统一等待
reqs = [dist.isend(send_buf, dst=peer), dist.irecv(recv_buf, src=peer)]
for r in reqs:
    r.wait()
```

> 【补充】实际框架里更推荐 `dist.batch_isend_irecv()` 一次性提交一批收发请求，由后端统一调度——既避免死锁又能重叠通信。Megatron-LM 的 PP 实现大量依赖这种批量异步收发。

2.3 七个集合原语：输入 → 输出

记号：共 N 个 rank。下面全部用 4 卡、可手算的小数据，**把每个原语的语义「看」出来**。

2.3.1 Broadcast：一份数据复制给所有人

```
操作前:              Broadcast 后:
GPU0: [1,2,3]        GPU0: [1,2,3]
GPU1: [ ]            GPU1: [1,2,3]
GPU2: [ ]            GPU2: [1,2,3]
GPU3: [ ]            GPU3: [1,2,3]
```

用途：训练开始时把 rank 0 的初始参数广播给所有进程，保证各卡起点一致（DDP 初始化就是这么做的）。

2.3.2 Scatter / Gather：分发与收集

```
Scatter（GPU0 分发）：            Gather（收集回 GPU0）：
操作前          操作后            操作前          操作后
GPU0:[a,b,c,d]  GPU0:[a]          GPU0:[a]        GPU0:[a,b,c,d]
GPU1:[ ]        GPU1:[b]          GPU1:[b]        GPU1:[b]
GPU2:[ ]        GPU2:[c]          GPU2:[c]        GPU2:[c]
GPU3:[ ]        GPU3:[d]          GPU3:[d]        GPU3:[d]
```

Scatter ≠ Broadcast：Broadcast 把同一份完整数据复制给所有卡；Scatter 把数据切开分发，每卡只拿到一段。早期 `DataParallel` 用 Scatter 分发 mini-batch、Gather 收集输出。

2.3.3 Reduce：规约到一个 rank

所有 rank 的数据按运算（求和/求最大）规约，**结果只放到根 rank**：

```
操作前:                Reduce(sum) 到 GPU0:
GPU0: [1,2,3]          GPU0: [10,20,30]
GPU1: [2,4,6]          GPU1: [2,4,6]    (不变)
GPU2: [3,6,9]          GPU2: [3,6,9]    (不变)
GPU3: [4,8,12]         GPU3: [4,8,12]   (不变)
```

语义上 `AllReduce = Reduce + Broadcast`（但实现上远比「先归约再广播」高效）。

2.3.4 AllReduce：规约结果人人有份（核心）

```
操作前:                AllReduce(sum) 后:
GPU0: [1,2,3]          GPU0: [10,20,30]
GPU1: [2,4,6]          GPU1: [10,20,30]
GPU2: [3,6,9]          GPU2: [10,20,30]
GPU3: [4,8,12]         GPU3: [10,20,30]
```

按元素对齐：`1+2+3+4=10`、`2+4+6+8=20`、`3+6+9+12=30`。操作后四张卡完全一致——这正是 DDP 里「所有卡梯度同步」的含义。

用途：DDP 同步梯度（求和后除以 N 取平均）；TP 合并切分矩阵乘的部分和。

2.3.5 ReduceScatter：规约后只留自己那份

```
操作前:                ReduceScatter(sum) 后:
GPU0: [1,2,3,4]        GPU0: [4]    (第 0 片求和: 1+1+1+1)
GPU1: [1,2,3,4]        GPU1: [8]    (第 1 片求和: 2+2+2+2)
GPU2: [1,2,3,4]        GPU2: [12]   (第 2 片求和: 3+3+3+3)
GPU3: [1,2,3,4]        GPU3: [16]   (第 3 片求和: 4+4+4+4)
```

（4 卡持相同向量只是为了让求和结果一目了然：每片被 4 卡各贡献一次，故第 i 片为 4×(i+1)。）

用途：ZeRO-2/3 与 FSDP 反向——梯度规约后每卡只需要自己负责更新的那一分片，用 ReduceScatter 一步到位。

2.3.6 AllGather：分片拼成完整数据

```
操作前:          AllGather 后:
GPU0: [A]        GPU0: [A,B,C,D]
GPU1: [B]        GPU1: [A,B,C,D]
GPU2: [C]        GPU2: [A,B,C,D]
GPU3: [D]        GPU3: [A,B,C,D]
```

它**不做规约，只做拼接**。用途：ZeRO-3 / FSDP 前向，计算前用 AllGather 临时拼出完整层参数。

> 把 2.3.5 和 2.3.6 的图叠起来看：ReduceScatter 让每卡「从完整数据收缩到一个求和分片」，AllGather 让每卡「从一个分片扩张回完整数据」，二者互为逆向。

2.3.7 All-to-All：人人给人人发不同数据

记 `D_ij` 表示「原本在 GPU i、要发给 GPU j」的块。操作后每卡收到所有卡发给它的数据，相当于把数据矩阵做了一次转置：

```
操作前(每卡按目的地排):          All-to-All 后(每卡收齐发给自己的):
GPU0: [D00,D01,D02,D03]          GPU0: [D00,D10,D20,D30]
GPU1: [D10,D11,D12,D13]          GPU1: [D01,D11,D21,D31]
GPU2: [D20,D21,D22,D23]          GPU2: [D02,D12,D22,D32]
GPU3: [D30,D31,D32,D33]          GPU3: [D03,D13,D23,D33]
```

第 j 列（所有卡发给 GPU j 的块）在操作后变成了 GPU j 的那一行——这就是「矩阵转置」的含义。用途：MoE 的 token dispatch/combine、Ulysses 在「序列维 ↔ Head 维」之间切换切分方式。

2.3.8 贯穿全模块的恒等式

```
AllReduce  =  ReduceScatter  +  AllGather
```

直观理解：先 ReduceScatter 让每卡拿到「自己那份的全局求和结果」，再 AllGather 把各卡分片拼回完整结果。这个拆解不只是理论恒等式，更是 Ring AllReduce 的实现蓝图，也是 ZeRO 能「用两个半步通信代替一次 AllReduce」的根本依据。

2.4 Ring AllReduce：为什么通信量与卡数无关

AllReduce 是训练里最高频的通信，它的效率直接决定 DDP 的扩展性。

2.4.1 朴素实现的带宽瓶颈

最直接的做法：所有卡把数据发给中心 rank，中心求和后广播回去。问题是**中心节点成了带宽瓶颈**——N 张卡的数据都要挤过它的一条链路。主卡收发量 `2(N-1)Ψ`，随 N 线性增长，无法扩展。

2.4.2 Ring 的两阶段

把 N 张卡逻辑上连成一个环，每张卡只和左右邻居通信。设每卡持有大小为 Ψ 的数据，先切成 N 个分片：

```
        ┌──── rank0 ────┐
        │               │
     rank3            rank1
        │               │
        └──── rank2 ────┘

阶段一 Reduce-Scatter（N-1 步）：每步发一个分片给下家，收上家的分片并累加
  → 结束后每卡有一个「全局求和完成」的分片（不同卡完成不同分片）

阶段二 All-Gather（N-1 步）：把已求和的分片沿环传递 N-1 步
  → 结束后每卡集齐全部 N 个求和后的分片 = 完整 AllReduce 结果
```

2.4.3 通信量推导（必会）

数据切成 N 份，每份 `Ψ/N`；两阶段各 N-1 步，每步每卡发送一个分片：

```
每卡发送量 = (N-1)·(Ψ/N) + (N-1)·(Ψ/N)  =  2·(N-1)/N · Ψ
             └ Reduce-Scatter ┘ └ All-Gather ┘
```

关键看系数：

```
2·(N-1)/N · Ψ  =  2Ψ·(1 − 1/N)  →(N 大)  2Ψ
```

当 N 较大时，每卡收发量趋近 2Ψ，与卡数无关。8 卡还是 1024 卡做 DDP，每卡每步通信量都稳定在约 2Ψ——这是 Ring 能支撑大规模数据并行的根本原因。接收量与发送量相同。

2.4.4 带宽最优性与拓扑选择

拓扑	每卡通信量	步数	适用
Ring AllReduce	`2(N-1)/N·Ψ ≈ 2Ψ`	2(N-1)，随 N 线性	大数据量（梯度同步）
Tree AllReduce	略高于 Ring	O(log N)	小数据量（控制信号、小张量）
Double Binary Tree	接近 Ring	O(log N)	大规模多机 AllReduce（NCCL 2.4+）

Ring 在大数据量下逼近带宽上限（带宽最优），但有 `2(N-1)` 步，**延迟随 N 线性增长**。所以小张量用 Tree（低延迟），大梯度用 Ring（高带宽）。实际中 NCCL 会按数据量、卡数、拓扑自动选，无需手动指定——这也是 `NCCL_DEBUG=INFO` 能看到「选了哪种算法」的原因。

2.5 通信量的量化方法

2.5.1 统一记号

符号	含义
Ψ	参数量（或待通信张量的元素数）
N	参与通信的 rank 数
α	单次通信的固定延迟
B	链路带宽（Bytes/s）
b, s, h	batch、序列长度、隐藏维
L	层数

2.5.2 三类原语的每卡通信量（Ring 实现，大 N 近似）

原语	每卡通信量	记忆法
AllReduce	2Ψ	ReduceScatter + AllGather
ReduceScatter	Ψ	AllReduce 的一半
AllGather	Ψ	AllReduce 的另一半

这张表解释了一个常见困惑：为什么 FSDP（ZeRO-3）每步是 3Ψ 而 DDP 只有 2Ψ？因为 FSDP 前向要 AllGather（Ψ）、反向要 AllGather（Ψ）+ ReduceScatter（Ψ），合计 3Ψ，比 DDP 多 50%。这个账在第 5 章会反复用到。

2.5.3 通信时间：α–β 模型

```
T ≈ α + 数据量/B
```

项	含义	何时主导	优化方向
α（延迟项）	建链、同步、握手，与数据量无关	小数据量	用 Tree、合并小消息、减少通信次数
数据量/B（带宽项）	数据量越大越慢	大数据量	用 Ring、降精度通信（bf16 reduce）、压缩

务必用对带宽：机内 NVLink（数百 GB/s）和机间 IB（约 50 GB/s）代入同一个公式会差十倍——这定量解释了「为什么 TP 必须放机内」。

【补充】一个具体的算例：7B 模型 DDP，梯度 2Ψ = 14 GB，N=8。 – 机内 NVLink，有效带宽按 300 GB/s 量级估【量级】：`T ≈ 2×14/300 ≈ 0.09 s` – 跨机 IB 400 Gbps（50 GB/s）：`T ≈ 2×14/50 ≈ 0.56 s` 一次迭代如果计算只要 0.3 s，跨机时通信就完全暴露了。这就是「带宽决定作用域」的算术版。

2.5.4 计算/通信重叠

通信不必「干等」。理想情况下如果通信能完全被计算盖住，通信几乎「免费」：

```
无重叠（串行）：   [── 计算 layer N ──][AllReduce][── layer N-1 ──][AllReduce]
有重叠（流水）：   [── 计算 layer N ──]
                          [AllReduce bucket1]
                    [── 计算 layer N-1 ──]
                          [AllReduce bucket2]
```

这就是 DDP 的 Bucket 机制、ZeRO 的 prefetch、FSDP 的 backward prefetch 的共同目标：把 `T_comm` 藏进计算时间。第 4、5 章会看到具体实现。

2.6 NCCL：环境变量与调优

NCCL（NVIDIA Collective Communications Library，读「nickel」）是 GPU 集合通信的事实标准。`backend="nccl"` 之后，AllReduce/AllGather 底层全由它执行。它会自动探测硬件拓扑（NVLink / PCIe / IB）并选择算法与路径。

问题在于：**NCCL 默认是个黑盒**。探测出错（选了没插网线的网卡、走了慢速 PCIe）时，训练会卡死或慢得反常，且没有报错。环境变量就是打开这个黑盒的窗口。

2.6.1 诊断类

变量	取值	作用
`NCCL_DEBUG`	`WARN`（默认近似）/ `INFO` / `TRACE`	排障第一招，`INFO` 打印拓扑、算法、网卡、通道
`NCCL_DEBUG_SUBSYS`	`NET,GRAPH,INIT,COLL`	缩小日志范围，如只看网络与拓扑
`NCCL_DEBUG_FILE`	路径模板	把日志写文件（多进程时避免刷屏）

`NCCL_DEBUG=INFO` 后典型日志：

```
NCCL INFO NET/IB : Using [0]mlx5_0:1/IB        # 用了哪块 IB 网卡
NCCL INFO Channel 00 : 0[0] -> 1[1] via NVLink # 卡间走 NVLink
NCCL INFO Ring 00 : 0 1 2 3 4 5 6 7            # Ring 连接顺序
```

重点看三件事：(1) 机间是否用上 IB（而不是退化到 socket）；(2) 机内卡间是否走 NVLink（而不是 PCIe）；(3) Ring/Tree 拓扑是否符合预期。

2.6.2 干预类（指定路径）

变量	示例	用途
`NCCL_SOCKET_IFNAME`	`eth0` / `^docker,lo`	指定以太网卡；`^` 表示排除
`NCCL_IB_HCA`	`mlx5_0,mlx5_1`	指定 IB HCA 设备
`NCCL_IB_DISABLE`	`1`	禁用 IB，强制走 socket（排障）
`NCCL_GID_INDEX`	`3`	RoCE 场景选 GID（v2/v1）
`NCCL_NET_GDR_LEVEL`	`0~5`	GPUDirect RDMA 启用层级

2.6.3 二分排障开关（用完必须关）

变量	关掉什么	若关掉后正常，说明
`NCCL_P2P_DISABLE=1`	GPU 间 P2P 直连（NVLink/PCIe P2P），改走主机内存中转	P2P 链路或驱动有问题
`NCCL_SHM_DISABLE=1`	机内共享内存（`/dev/shm`）通信，强制走网络	`/dev/shm` 太小（容器默认 64MB）

这两个开关**正常训练时不要开**——它们都会显著降低性能（放弃最快的通信路径）。正确用法是「二分法」：开启后若问题消失，就锁定了故障层；定位后立刻关掉。

2.6.4 性能调优类

变量	默认	调整场景
`NCCL_ALGO`	自动	强制 `Ring` / `Tree` / `CollNet`，验证算法假设
`NCCL_PROTO`	自动	`LL`（低延迟）/ `LL128` / `Simple`（高带宽）
`NCCL_MIN_NCHANNELS` / `NCCL_MAX_NCHANNELS`	自动	增加通道数压榨带宽（小消息多时调小）
`NCCL_BUFFSIZE`	约 4 MB 量级【量级】	大消息增大缓冲
`NCCL_NSOCKS_PERTHREAD` / `NCCL_SOCKET_NTHREADS`	自动	多网卡聚合、socket 带宽不足时调

> 【补充】调优的正确顺序是：先 `NCCL_DEBUG=INFO` 确认走了对的链路 → 再用 nccl-tests 量出实际带宽（第 12 章与速查文档给命令）→ 最后才动 `NCCL_ALGO` / `NCCL_PROTO` 这类旋钮。**先测后调，别盲调**。

2.6.5 一个可复现的验证实验

```
# collective_demo.py —— torchrun --nproc_per_node=4 collective_demo.py
import torch, torch.distributed as dist

def main():
    dist.init_process_group(backend="nccl")
    rank, ws = dist.get_rank(), dist.get_world_size()
    torch.cuda.set_device(rank)
    dev = torch.device(f"cuda:{rank}")

    # 1) AllReduce(sum): 每卡持 rank 值，求和后全为 0+1+2+3=6
    x = torch.tensor([float(rank)], device=dev)
    dist.all_reduce(x, op=dist.ReduceOp.SUM)
    print(f"[AllReduce] rank{rank} -> {x.item()}")

    # 2) AllGather: 每卡一个分片，收集后都拿到 [0,1,2,3]
    src = torch.tensor([float(rank)], device=dev)
    out = [torch.zeros(1, device=dev) for _ in range(ws)]
    dist.all_gather(out, src)
    print(f"[AllGather] rank{rank} -> {[t.item() for t in out]}")

    # 3) ReduceScatter: 每卡提供 ws 个分片，规约后只留自己那份
    inputs = [torch.tensor([float(rank * 10 + i)], device=dev) for i in range(ws)]
    out = torch.zeros(1, device=dev)
    dist.reduce_scatter(out, inputs, op=dist.ReduceOp.SUM)
    print(f"[ReduceScatter] rank{rank} -> {out.item()}")

    dist.destroy_process_group()

if __name__ == "__main__":
    main()
```

```
NCCL_DEBUG=INFO torchrun --nproc_per_node=4 collective_demo.py 2>&1 \
  | grep -iE "Ring|Tree|Algorithm|via"
```

进阶练习（原文建议）：把张量从 1 个元素加大到几百 MB，用 `torch.cuda.Event` 计时，算实测带宽。你会看到小数据量时延迟主导、大数据量时逼近链路带宽，正好印证 `T ≈ α + 数据量/B`。

2.7 本章小结与自检

1. 并行 = 切分 + 通信；切分决定显存，通信决定速度。

2. 七个原语的语义必须能默画：Broadcast 复制、Scatter/Gather 分发收集、Reduce 规约到一点、AllReduce 规约给所有人、ReduceScatter 规约留分片、AllGather 拼接、All-to-All 全交换。
3. `AllReduce = ReduceScatter + AllGather` 既是 Ring 的实现蓝图，也是 ZeRO 通信分析的基础。
4. Ring 每卡通信量 `2(N-1)/N·Ψ ≈ 2Ψ`，**与卡数无关**；小数据量用 Tree 降延迟。
5. `T ≈ α + 数据量/B`，代入正确带宽（机内数百 GB/s vs 机间约 50 GB/s）。
6. 排障顺序：`NCCL_DEBUG=INFO` → 看链路 → 二分开关隔离 → 定位后恢复默认。

自检题：

1. 手推 Ring AllReduce 的 `2(N-1)/N·Ψ`，并说明「与 N 无关」在工程上意味着什么。
2. 一个 8 卡任务，AllReduce 一个 100 MB 的 BF16 梯度，机内 300 GB/s、机间 50 GB/s，分别要多久？
3. 为什么 `ReduceScatter` 之后还要 `AllGather` 才能得到 AllReduce 的结果？中间每卡各拿到什么？
4. 你的训练在容器里 hang 住，`NCCL_DEBUG=INFO` 显示退化到 socket。给出至少三条排查方向。

第 3 章 优化器与显存：12Ψ 是怎么来的

> 对应 AIInfraGuide 模块三 第 3 章（`第3章-优化器` 扉页 + `31-优化器原理与显存开销分析` 正文）。

3.1 优化器在训练循环里的位置

反向传播只告诉你参数该往哪个方向走，「走多远、怎么走」全靠优化器。它在循环里的位置非常固定：

```
前向算 loss → loss.backward() 得到梯度 g → optimizer.step() 更新参数 → zero_grad() → 回到前向
```

`optimizer.step()` 一行背后做三件事：读梯度、结合优化器**自己维护的内部状态**算更新量、把更新量应用到参数上。

> 关键就在「内部状态」四个字。这些状态在 `step()` 之间**持久驻留显存**，不是算完就丢的临时变量。这是优化器成为显存大户的根源。

3.2 演进主线：每加一样东西，多存一份状态

优化器	更新用到的信息	每参数额外状态	7B 模型的优化器开销	典型场景
SGD	当前梯度	0 份（仅 FP32 参数副本 4B）	~28 GB	简单凸问题、教学
Momentum SGD	梯度 + 一阶动量	1 份（m，8B 含副本）	~56 GB	CV 里的 SGD+Momentum
Adam	梯度 + 一阶 + 二阶	2 份（m, v，12B 含副本）	~84 GB	通用默认
AdamW	同 Adam，解耦 weight decay	2 份（12B）	~84 GB	**大模型训练标准**

公式对照（原文给出）：

```
SGD:       θ ← θ − η·g
Momentum:  m ← β·m + g ;  θ ← θ − η·m
Adam:      m ← β₁m + (1−β₁)g
           v ← β₂v + (1−β₂)g²
           θ ← θ − η · m̂ / (√v̂ + ε)      m̂, v̂ 为偏差修正后的 m, v
AdamW:     先按 Adam 更新，再做独立的一步 θ ← θ − η·λ·θ
```

AdamW 到底改了什么：原始 Adam 把 weight decay 塞进梯度里（`g ← g + λθ`），结果它和自适应学习率纠缠在一起——分母 `√v̂` 会把不同参数的正则化力度扭曲得强弱不一。AdamW 把 weight decay 拿出来做成独立的参数缩放，与梯度更新彻底解耦。就这一个改动，让它成了当今大模型的事实标准。

> 偏差修正为什么必要：`m₀ = v₀ = 0` 会让初期估计偏小，需要除以 `1-β^t` 修正。这一步在实现里是隐式的，但它解释了「为什么 warmup 阶段的有效步长和你算的不一样」。

3.3 显存账：12 字节/参数

以混合精度训练为前提（前向/反向 BF16，优化器内部 FP32）：

优化器	额外状态	每参数优化器显存	拆解
SGD	无	4 B	仅 FP32 参数副本
Momentum	一阶动量	8 B	FP32 副本 + FP32 m
Adam / AdamW	一阶 + 二阶	**12 B**	FP32 副本 4 + m 4 + v 4

代入 7B：

```
12 × 7×10⁹ ≈ 84 GB   ← 已超过一张 80GB 卡
```

完整账本（原文的百分比表）：

显存构成	大小	占比
优化器状态（FP32 主权重 + m + v）	84 GB	**75%**
BF16 梯度	14 GB	12.5%
BF16 参数	14 GB	12.5%

这张账直接解释了 ZeRO 的设计动机：**第一刀就砍优化器状态**，因为它是占比最大的一块，切它性价比最高。而 DDP 里每张卡都各存一份完整的 84 GB，冗余到了极点。

3.3.1 为什么优化器状态必须 FP32

一个常见疑问：既然前向反向都用 BF16 了，优化器为什么不也用 BF16 省一半？

答案藏在「更新量太小」这件事上：

1. 每步的参数更新量约等于「学习率 × 梯度」，量级常在 10^{-4} 甚至更小。
2. BF16 只有 **7 位尾数**，相对精度约 $2^{-8} \approx 0.4\%$。表示不了这么小的变化——加到较大的参数上等于没加（**大数吃小数**）。
3. 训练要跑成千上万步，每步的舍入误差不断累积，最终导致训练发散。

所以混合精度训练必须保存一份 **FP32 master weights**：每步用 FP32 完成更新，再把结果转成 BF16 供下一次前向使用。这份副本就是账本里那 28 GB 的来源，**省不掉**。

精度	指数位	尾数位	相对精度	能否承载 1e-4 级更新
FP32	8	23	~1e-7	✅
TF32	8	10	~1e-3	⚠️ 勉强（多用于矩阵乘而非权重存储）
BF16	8	7	~4e-3	❌
FP16	5	10	~1e-3	❌（且范围小易溢出）

3.4 大 Batch 优化器：LARS 与 LAMB

数据并行有个直接副作用——**有效 Batch 被线性放大**。N 卡数据并行，全局 batch 就是单卡的 N 倍；卡到几十上百张时有效 batch 轻松上万。

大 batch 的好处是梯度方差更小、方向更「干净」，理论上能配更大学习率。但实践中直接线性放大学习率，训练往往一上来就震荡甚至发散。更麻烦的是**模型不同层的参数尺度差异极大**（Embedding 与 LayerNorm 根本不是一个量级），一个全局学习率怎么调都顾此失彼。

> 在动用 LARS/LAMB 之前，工程上先试两招更简单的：**线性缩放**（batch 放大 k 倍，lr 也放大 k 倍）和 **warmup**（前几百到几千步线性爬升）。这两招能覆盖大多数中等规模场景。

优化器	基础	缩放粒度	主要战场
LARS	SGD + Momentum	逐层（参数范数 / 梯度范数）	ResNet 等 CV 大 batch
LAMB	Adam	逐层（信赖域 × Adam 更新）	BERT/Transformer 大 batch

```
LARS:  λ_l = ||θ_l|| / ||g_l||        # 参数大而梯度小的层放开步子，反之收紧
LAMB:  r_l = Adam 更新量；再乘 φ(||θ_l||)/||r_l|| 约束幅度
```

LAMB 最著名的战绩：把 BERT 预训练的 batch size 拉到 65536 还能稳定收敛，训练时间从 3 天压到 76 分钟【原文】。

✅ 什么时候该换 LAMB：数据并行卡数很多（64+）、有效 batch 极大、AdamW 已经压不住训练不稳时。**卡数不多、batch 适中就老老实实用 AdamW**——别为了用而用。

3.5 动手：把优化器的账本翻出来看

```
import torch, torch.nn as nn

model = nn.Linear(1000, 1000)          # 1,001,000 个参数
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

x = torch.randn(8, 1000)
loss = model(x).sum()
loss.backward()
optimizer.step()                        # 跑一步，Adam 才真正分配 m、v

state_elems = 0
for p, st in optimizer.state.items():
    for k, v in st.items():
        if torch.is_tensor(v):
            state_elems += v.numel()
            print(f"  状态 '{k}': {v.numel():,} 个元素, dtype={v.dtype}")
print(f"优化器状态总元素: {state_elems:,} (约等于 2 × 参数量)")
```

会看到 Adam 存了 `exp_avg`（m）和 `exp_avg_sq`（v）两份，元素数都等于参数量。换成不带 momentum 的 `SGD`，state 几乎是空的——这就是 SGD 省显存的原因。

把「存几份」变成估算工具：

```
def estimate_memory_gb(n_params_billion, optimizer="adamw", dtype_bytes=2):
    """混合精度训练的静态显存（GB），不含激活值"""
    P = n_params_billion * 1e9
    param = P * dtype_bytes                          # BF16 参数
    grad = P * dtype_bytes                           # BF16 梯度
    state_copies = {"sgd": 0, "momentum": 1, "adamw": 2}[optimizer]
    optim = P * 4 * (1 + state_copies)               # FP32 主权重 + 各状态
    total = (param + grad + optim) / 1e9
    print(f"{optimizer} @ {n_params_billion}B: 参数{param/1e9:.0f} + 梯度{grad/1e9:.0f} "
          f"+ 优化器{optim/1e9:.0f} = {total:.0f} GB，优化器占 {optim/(param+grad+optim):.0%}")
    return total

estimate_memory_gb(7, "adamw")     # 参数14 + 梯度14 + 优化器84 = 112 GB，优化器占 75%
estimate_memory_gb(70, "adamw")    # 参数140 + 梯度140 + 优化器840 = 1120 GB
```

把 `optim` 除以并行度 N，就是 ZeRO-1 的效果——这是下一步分析 ZeRO 收益的基本功。

3.6 本章小结与自检

1. 演进主线 = 不断加状态换稳定性：SGD 0 份、Momentum 1 份、Adam/AdamW 2 份。
2. AdamW 每参数 12 字节，占「参数+梯度+优化器」静态显存的 75%——ZeRO-1 优先切它的理由。
3. 优化器状态必须 FP32（master weights），否则 `1e-4` 级更新会被 BF16 的 7 位尾数吃掉。
4. 大 batch 失稳先用线性缩放 + warmup；卡数 64+ 且压不住时才上 LAMB。
5. `estimate_memory_gb` 稍加改造（把 optim 除以 N）就能估 ZeRO 的显存收益。

自检题：

1. 手算 13B 模型在 AdamW + BF16 下的优化器状态显存，以及它在静态显存里的占比。
2. 如果把 master weights 换成 TF32（19 位，按 4 字节存），显存会变吗？精度会怎样？
3. 为什么 DDP 下「每卡的 84GB 优化器状态」是纯冗余？ZeRO 用什么办法消除它而不改变数学结果？
4. 8 卡数据并行把 batch 从 32 提到 256，学习率该怎么调？如果调到 8 倍后发散，下一步做什么？

第 4 章 数据并行：从 DP 到 DDP

> 对应 AIInfraGuide 模块三 第 4 章（`第4章-数据并行` 扉页 + `41-数据并行详解` 前半 + `42-pytorch-数据并行从原理到实战` 前半）。FSDP 与 ZeRO 放在第 5 章。

4.1 数据并行解决的是「太慢」

一个班 6000 份卷子，一位老师批一周；找 6 位老师各批 1000 份，一天收工。前提是**每位老师手里都有一份完整的评分标准**（完整模型副本），批完还要**对齐评分尺度**（同步梯度）。

技术版本的五步：

1. 每块 GPU 持有一份完整模型副本（参数完全相同）。
2. 把一个大 batch 均匀切到各 GPU，每卡只算自己那份（local batch）。
3. 各卡独立前向 + 反向，算出各自的梯度。
4. 通信把所有卡的梯度**取平均**，让每卡拿到一致的梯度。
5. 每卡用这份一致的梯度更新——更新后所有副本依然相同。

> 核心约束：**每卡都要装得下完整模型**。DP 加速了训练，但没有减少单卡的显存负担。

4.2 数学基础：为什么必须「取平均」而不是「求和」

全局 batch `B`，损失是样本损失的平均：

$$L = (1/B) \cdot \Sigma_i \ell(x_i; \theta) \qquad g = \nabla L = (1/B) \cdot \Sigma_i \nabla \ell(x_i; \theta)$$

把 B 个样本分给 N 卡，每卡 `B/N` 个。第 k 卡的本地梯度（内部也做了平均）：

$$g_k = (N/B) \cdot \Sigma_{i \in D_k} \nabla \ell(x_i; \theta)$$

对 N 个本地梯度取平均：

$$\bar{g} = (1/N) \cdot \Sigma_k g_k = (1/N) \cdot \Sigma_k (N/B) \cdot \Sigma_{i \in D_k} \nabla \ell = (1/B) \cdot \Sigma_i \nabla \ell = g$$

结论：N 卡数据并行的平均梯度精确等于单卡在全局 batch 上的梯度。 这不是近似技巧，是严格的数学等价（同步 SGD、各卡 local batch 等大时）。

两个实践推论：

推论	说明
必须 AllReduce 求平均	每卡本地损失已在 local batch 内平均过，跨卡再平均 N 份才还原全局平均。**如果求和，等价于把学习率放大 N 倍**
有效 batch = local batch × N	这是扩卡时要调学习率的根本原因

> 等价性只在「同步梯度 + 各卡 local batch 等大」时成立。最后一个 batch 不满、或异步更新，等价性被打破。

4.3 第一代 DataParallel：单进程多线程的三个致命缺陷

`nn.DataParallel` 的卖点是「改一行就能多卡」，但它是**单进程多线程**架构，官方已不推荐。

```
主卡 GPU0（持有模型 + 输入 batch）
   │
   ├─ Scatter 输入到各卡 → 各卡复制模型副本 → 前向
   │
   └─ Gather 输出回主卡 → 主卡算 loss + 反向 → 各卡梯度汇总回主卡 → 主卡更新
```

缺陷	原因
GIL 限制	单进程多线程，Python 全局解释器锁让调度被串行化，CPU 侧成瓶颈
负载不均	主卡额外承担 Scatter/Gather、loss 计算、梯度汇总，显存和算力都比别人重
通信低效	每步都要重新复制模型；数据走主卡中转，容易挤在 PCIe 而非 NVLink

> DP 最常见的报错就是「主卡 OOM 而其他卡显存还很空」。病根是**单进程 + 主卡中心化**；DDP 的所有改进都在拆这两个前提。

4.4 第二代 DDP：每卡一进程 + 去中心化 AllReduce

维度	DP	DDP
进程模型	单进程多线程	每卡一个进程，无 GIL 争抢
模型副本	每步重新 replicate	只在初始化时 Broadcast 一次
梯度同步	汇总到主卡求和	去中心化 AllReduce，各卡对等
负载	主卡偏重	各卡均衡
通信路径	经主卡中转	点对点 Ring，充分利用 NVLink

DDP 的生命周期 = 一次广播 + 多次同步：

```
初始化: Broadcast(rank0 参数 → 所有 rank)      ← 只通信一次
前向:   各算各的，零通信
反向:   AllReduce 梯度（按 bucket 流水化）      ← 每步一次
更新:   各卡用一致的梯度独立更新（结果仍一致）
```

4.4.1 Bucket 机制：把通信藏进计算

最朴素的做法是「等整个反向算完，再统一 AllReduce」——但这样通信时 GPU 干等。DDP 的巧思是利用反向的顺序性：**反向是从最后一层往前逐层算的，靠后的层梯度先就绪**。

于是 DDP 把梯度按反向顺序打包成若干 **Bucket**（默认约 25 MB 一个）：

```
规则：某个 bucket 里所有梯度都就绪 → 立即对该 bucket 发起 AllReduce
      与此同时，更靠前的层还在继续算反向
```

```
时间 →
反向 layer N    [████████]
反向 layer N-1            [████████]
反向 layer N-2                    [████████]
AllReduce b1       [====]            ← 与 layer N-1 计算重叠
AllReduce b2                 [====]   ← 与 layer N-2 计算重叠
```

旋钮	默认	调大	调小
`bucket_cap_mb`	约 25 MB	通信次数少、单次开销占比低；但重叠变差（要等更久才凑满）	重叠好、但次数多、固定开销占比高

4.4.2 DDP 的四个常用参数

```
model = DDP(
    model,
    device_ids=[local_rank],
    output_device=local_rank,
    bucket_cap_mb=25,                  # 桶大小
    find_unused_parameters=False,      # 有参数不参与前向时才开
    gradient_as_bucket_view=True,      # 梯度直接指向 bucket，省一份显存
    static_graph=False,                # 计算图固定（如无分支）可开，优化 bucket 顺序
)
```

参数	作用	什么时候动
`find_unused_parameters`	让 DDP 主动标记未参与前向的参数	有条件分支/多任务头时**必须**开，否则对应 bucket 永远等不满 → 死锁。但**有额外开销，能避则避**
`gradient_as_bucket_view`	梯度视图直接指向 bucket 内存	默认建议开，省一份梯度显存
`static_graph`	告知计算图固定，DDP 可静态规划 bucket	图确实固定时开，能提升重叠效果
`broadcast_buffers`	是否每步广播 BN 的 buffer	有 BN 时保持 True；纯 Transformer 可设 False 省一次通信

4.4.3 DDP 的扩展性天花板

计算/通信重叠让 DDP 在中等规模下几乎线性加速，但红利在超大规模失效：

- 随着 GPU 数增加到数百上千，协调开销和网络需求显著增长，通信逐渐盖不住计算；
- 原文给出的工程观察：**512+ GPU 规模下，DP 通信开始受限于环延迟（ring latency）**——信号绕 Ring 传播一圈的时间——无法再被完全重叠。到了这个点，应该转向其他并行维度（TP/PP/ZeRO），而不是继续堆 DP。

⚠️ 注意口径：加卡只摊薄计算时间，**单卡显存占用保持恒定**——这正好印证「DP 不省显存」。

4.5 DDP 的根本局限与通信账

DDP 数学严格等价、实现简单，但它继承了数据并行的天生约束：每卡都要装下完整的「参数 + 梯度 + 优化器」。

```
FP16/BF16 参数       2Ψ
FP16/BF16 梯度       2Ψ
FP32 优化器状态     12Ψ   (master 4 + m 4 + v 4)
──────────────────────────
合计                16Ψ   → 7B ≈ 112 GB，单张 80GB 装不下
```

无论加多少卡，DDP 都救不了这个数字，因为每卡的这份账是完全冗余的。要突破显存墙，就要问：这份 16Ψ 的冗余能不能也切开分给各卡？——这是第 5 章 FSDP/ZeRO 的出发点。

4.5.1 每步 2Ψ 的出处（复习第 2 章）

```
Ring AllReduce 每卡发送量 = 2·(N−1)/N·Ψ ≈ 2Ψ
```

方案	单卡通信量	是否随 N 膨胀
中心化汇总（DP 式）	主卡 `2(N−1)Ψ`	❌ 主卡线性膨胀，成瓶颈
Ring AllReduce（DDP）	`2(N−1)/N·Ψ ≈ 2Ψ`	✅ 恒定

> 2Ψ 是理想 Ring 下的带宽下界。实际还受拓扑（NVLink / PCIe / IB）、消息大小、NCCL 算法选择影响。跨机时 IB 带宽远低于机内，往往成为整体瓶颈。

4.6 一个正确的 DDP 训练脚本（骨架）

```
# ddp_demo.py  —— torchrun --nproc_per_node=4 ddp_demo.py
import os, torch, torch.nn as nn, torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data import DataLoader, TensorDataset
from torch.utils.data.distributed import DistributedSampler

def setup():
    rank = int(os.environ["RANK"]); local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)          # ① 先绑卡（用 local_rank！）
    dist.init_process_group(backend="nccl")    # ② 后建组
    return rank, local_rank

def main():
    rank, local_rank = setup()
    device = torch.device(f"cuda:{local_rank}")

    ds = TensorDataset(torch.randn(2048, 128), torch.randint(0, 10, (2048,)))
    sampler = DistributedSampler(ds, num_replicas=dist.get_world_size(),
                                 rank=rank, shuffle=True)
    loader = DataLoader(ds, batch_size=32, sampler=sampler)

    model = nn.Sequential(nn.Linear(128, 256), nn.ReLU(), nn.Linear(256, 10)).to(device)
    model = DDP(model, device_ids=[local_rank])          # ③ 包 DDP
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
    crit = nn.CrossEntropyLoss()

    for epoch in range(5):
        sampler.set_epoch(epoch)                         # ④ 每 epoch 重设种子
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            opt.zero_grad()
            loss = crit(model(xb), yb)
            loss.backward()                              # DDP 在此自动 AllReduce
            opt.step()
        if rank == 0:                                    # ⑤ 仅主进程干活
            print(f"[epoch {epoch}] loss={loss.item():.4f}")

    if rank == 0:
        torch.save(model.module.state_dict(), "model.pt")  # 注意 .module
    dist.destroy_process_group()

if __name__ == "__main__":
    main()
```

4.6.1 五个必须记住的细节

细节	正确做法	错误的后果
顺序	先 `set_device(local_rank)` 再 `init_process_group`	多进程都落到 0 号卡 → 显存爆

细节	正确做法	错误的后果
搬卡	先 `.to(device)` 再 `DDP(...)`	在 CPU 上包 DDP 再搬卡会出错
sampler	用了 `DistributedSampler` 就别再设 `shuffle=True`	冲突报错 / 数据重复
`set_epoch`	每 epoch 开头调 `sampler.set_epoch(epoch)`	每轮数据切分完全一样，shuffle 形同虚设（极隐蔽的 bug）
保存	`model.module.state_dict()`	key 带 `module.` 前缀，单卡加载对不上

4.7 本章小结与自检

1. 数据并行 = 复制模型 + 切数据 + 梯度平均。数学上与单卡全局 batch 严格等价；**必须平均，不能求和**。
2. DP 死于单进程 + 主卡中心化（GIL、负载不均、通信低效）；DDP 用「每卡一进程 + 去中心化 AllReduce + Bucket 重叠」取代。
3. DDP 每步通信量 `2Ψ`（Ring AllReduce），与卡数无关；512+ 卡时受环延迟限制，应换维度。
4. DDP 单卡显存恒为 `16Ψ`，加卡不减负——这是它的天花板，也是 FSDP 的动机。
5. 五个易错点：绑定用 local_rank、先搬卡再包 DDP、sampler 与 shuffle 二选一、set_epoch 别漏、保存用 `.module`。

自检题：

1. 推导「N 卡平均梯度 = 单卡全局 batch 梯度」，并解释为什么求和等价于 lr 放大 N 倍。
2. DDP 的 bucket 为什么能重叠？如果模型有大量参数不参与前向，会发生什么？怎么解？
3. 8 卡 DDP 训一个 3B 模型，单卡显存大概多少？够不够 80GB？
4. 为什么 DDP 里「初始化只广播一次，而梯度每步都要同步」？

第 5 章 ZeRO 与 FSDP：把 16Ψ 切成 1/N

> 对应 AIInfraGuide 模块三 第 5 章（`第5章-zero系列` 扉页）、第 4 章 FSDP 部分（`41-数据并行详解` §6–§9、`42-pytorch-数据并行从原理到实战` §3）。

5.1 冗余在哪里

DDP 之后，N 卡集群里每张卡都存了一份**完全相同**的 16Ψ（2 参数 + 2 梯度 + 12 优化器）。这意味着：

```
总存储 = N × 16Ψ          其中真正必要的信息只有 16Ψ
浪费比例 = (N-1)/N          8 卡 → 87.5% 的存储是冗余的
```

ZeRO 的核心洞察就是：**既然平时用不到别人的那份，为什么要存？** 范式是「**切分–聚合**」——平时每卡只存 1/N，需要时通过通信获取完整数据，用完即弃。

三阶段递进，从最容易切的（优化器状态）到最难切的（参数）：

阶段	切什么	每卡显存（Ψ 参数，N 卡）	每步每卡通信	对应 FSDP 策略
DDP 基线	不切	16Ψ	2Ψ	`NO_SHARD`
ZeRO–1	优化器状态	`4Ψ + 12Ψ/N`	≈ 2Ψ + AllGather	—（FSDP1 无直接对应）
ZeRO–2	+ 梯度	`2Ψ + 14Ψ/N`	≈ 2Ψ（RS + AG）	`SHARD_GRAD_OP`
ZeRO–3	+ 参数	`16Ψ/N`	**3Ψ**	`FULL_SHARD`

> 7B、N=8：DDP 112 GB → ZeRO–2 约 26 GB → ZeRO–3 = 14 GB。**这就是「单卡装不下」被解决的算术过程。**

5.2 ZeRO–1：切优化器状态

项	内容
切什么	Adam 的 FP32 主权重 + 一阶动量 + 二阶动量（共 12Ψ）
每卡显存	`2Ψ + 2Ψ + 12Ψ/N = 4Ψ + 12Ψ/N`
通信模式	梯度仍需 AllReduce（与 DDP 相同）；参数更新后需 AllGather 同步参数
通信量	与 DDP 接近（2Ψ）+ 一次参数 AllGather（Ψ）
适用	优化器状态是显存大头时（Adam 占 75%）

```
ZeRO-1 一步：
  ① 反向 → 每卡有完整梯度（2Ψ）
  ② AllReduce 梯度（与 DDP 一样）
  ③ 每卡只对「自己负责的 1/N 参数」做 FP32 更新（用自己的优化器状态分片）
  ④ AllGather 更新后的参数分片 → 每卡拿到完整参数
```

> 注意第 ③ 步：因为每卡只更新 1/N 的参数，所以只需要那 1/N 的优化器状态——这就是切分的合法性来源。

5.3 ZeRO-2：连梯度也切

项	内容
切什么	在 ZeRO-1 基础上，梯度也按分片存储
每卡显存	`2Ψ + 14Ψ/N`（2Ψ 参数 + 2Ψ/N 梯度 + 12Ψ/N 优化器）
通信模式	反向时用 **ReduceScatter** 替代 AllReduce，每卡只保留自己分片的聚合梯度
通信量	与 DDP 同量级：ReduceScatter(Ψ) + AllGather(Ψ) = 2Ψ
适用	模型中等偏大，切掉优化器+梯度后单卡能装下参数

> 原文的一句话点出了关键：「ReduceScatter = Ψ（比 AllReduce 的 2Ψ 少一半，但丢失了 AllReduce 隐含的 AllGather）」——省下的那 Ψ 正是后面要补的 AllGather。**ZeRO-2 不是免费省一半通信，而是把一次 AllReduce 拆成两半，只用其中一半。**

5.4 ZeRO-3：连参数也切

项	内容
切什么	参数、梯度、优化器状态全部切分
每卡显存	`16Ψ/N`（理想线性缩放）
通信模式	前向：AllGather 当前层参数 → 计算 → 释放；反向：AllGather 参数 → 算梯度 → ReduceScatter 梯度 → 释放
通信量	**3Ψ**（前向 AG Ψ + 反向 AG Ψ + RS Ψ），比 DDP 多 50%
适用	模型参数本身单卡装不下，必须分片

```
ZeRO-3 一个 Transformer Block 的生命周期：

  初始：每卡只存 1/N 参数分片
    │
    ├─ 前向 AllGather  → 临时拼出完整参数 → 前向计算 → 释放非本地分片
    │
    ├─ 反向 AllGather  → 再拼一次（前向后已释放）→ 算梯度
    │
    ├─ ReduceScatter   → 梯度聚合 + 分片回各卡（只留自己那片）
    │
    └─ 更新：每卡只更新自己负责的 1/N 参数和优化器状态
```

为什么是 3Ψ 而不是 2Ψ（第 2 章的恒等式直接给出答案）：

```
DDP:   AllReduce          = RS(Ψ) + AG(Ψ)              = 2Ψ
FSDP:  AG(前向) + AG(反向) + RS(反向)                    = 3Ψ
       └ 多了前向那一次 AllGather，因为前向后参数被释放了
```

5.5 FSDP：PyTorch 的原生实现

FSDP（FullyShardedDataParallel）是 PyTorch 对 ZeRO 思想的原生实现。它按 **FSDP unit**（通常一个 Transformer Block）为单位做「按需组装、用完即弃」。

策略	分片内容	显存节省	通信开销	适用
`FULL_SHARD`	参数 + 梯度 + 优化器（≈ ZeRO-3）	最高	最高（3Ψ）	大模型、显存紧张
`SHARD_GRAD_OP`	梯度 + 优化器，参数不分片（≈ ZeRO-2）	中等	中等	中等模型、想省通信
`HYBRID_SHARD`	机内 FULL_SHARD + 机间 DP	高	机间较低	**多机训练首选**
`NO_SHARD`	不分片（等价 DDP）	无	最低	调试对照

`HYBRID_SHARD` 的洞察值得单独说：机内有高带宽 NVLink，适合做通信密集的 FULL_SHARD；机间只有较慢的 IB，就退化成通信量小的数据并行（AllReduce）。**把高频通信关在机内，低频通信才跨机。**

5.5.1 FSDP 训练代码

```
import functools, torch, torch.nn as nn, torch.distributed as dist
from torch.distributed.fsdp import (
    FullyShardedDataParallel as FSDP, MixedPrecision, ShardingStrategy,
)
from torch.distributed.fsdp.wrap import size_based_auto_wrap_policy, transformer_auto_wrap_

def main():
    dist.init_process_group(backend="nccl")
    local_rank = int(__import__("os").environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)
    rank, world_size = dist.get_rank(), dist.get_world_size()

    model = MyTransformer().to(local_rank)

    bf16 = MixedPrecision(param_dtype=torch.bfloat16,
                          reduce_dtype=torch.bfloat16,
                          buffer_dtype=torch.bfloat16)

    # 关键：auto_wrap_policy 决定「分片单元」的大小
    wrap_policy = functools.partial(size_based_auto_wrap_policy,
                                    min_num_params=1_000_000)

    model = FSDP(
        model,
        sharding_strategy=ShardingStrategy.FULL_SHARD,
        mixed_precision=bf16,
        auto_wrap_policy=wrap_policy,
        device_id=local_rank,
        backward_prefetch=FSDP.BackwardPrefetch.BACKWARD_PRE,   # 预取下一层分片
        limit_all_gathers=True,                                  # 限制同时在途的 AG
    )

    opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
    sampler = DistributedSampler(ds, num_replicas=world_size, rank=rank, shuffle=True)
    loader = DataLoader(ds, batch_size=64, sampler=sampler, num_workers=2)

    for epoch in range(5):
        sampler.set_epoch(epoch)
        for xb, yb in loader:
            xb, yb = xb.to(local_rank), yb.to(local_rank)
            opt.zero_grad()
            loss = criterion(model(xb), yb)
            loss.backward()
            opt.step()
        if rank == 0:
            print(f"epoch {epoch} done")

    dist.destroy_process_group()
```

5.5.2 三个必须会的 FSDP 细节

细节	做法	说明
分片单元	`auto_wrap_policy`（`size_based` / `transformer_auto_wrap_policy`）	单元太大 → AllGather 峰值高（接近不分片）；太小 → 通信次数爆炸。**按 Transformer Block 切是最常用的选择**
保存	用 `torch.distributed.checkpoint` + `get_state_dict` / `set_state_dict`	直接 `torch.save(model.state_dict())` 可能只落盘本地分片；DCP 支持 **resharding**（8 卡存、4 卡读）
混合精度	`MixedPrecision(param/reduce/buffer_dtype)`	FSDP 的 AMP 必须显式配置，不能用外层 `autocast` 包住整个 forward

```
import torch.distributed.checkpoint as dcp
from torch.distributed.checkpoint.state_dict import get_state_dict, set_state_dict

# 保存（所有 rank 参与）
model_sd, optim_sd = get_state_dict(model, opt)
dcp.save({"model": model_sd, "optim": optim_sd}, checkpoint_id="./ckpt/epoch_5")

# 加载（所有 rank 参与）
model_sd, optim_sd = get_state_dict(model, opt)
dcp.load({"model": model_sd, "optim": optim_sd}, checkpoint_id="./ckpt/epoch_5")
set_state_dict(model, opt, model_state_dict=model_sd, optim_state_dict=optim_sd)
```

5.5.3 FSDP2：`fully_shard`

PyTorch 正在推进新一代 API（俗称 FSDP2），用 **per-parameter 分片**取代 FSDP1 的 module-wrapper，底层基于 **DTensor**：

```
from torch.distributed.fsdp import fully_shard

for layer in model.layers:
    fully_shard(layer)
fully_shard(model)
```

维度	FSDP1	FSDP2
分片粒度	整个 module 打包	单个参数（per-parameter）
底层抽象	FlatParameter	DTensor
与 TP 组合	困难	接口统一，是搭 2D/3D 的基础

维度	FSDP1	FSDP2
成熟度	稳定，存量生产首选	已可用、官方主推，但仍在快速迭代

> 【补充】FSDP2 的 `fully_shard` 与 TP（`parallelize_module` + DTensor）共享同一套 DTensor 抽象，这是它能「和 TP 组合」的根本原因；FSDP1 的 FlatParameter 会把参数摊平，破坏 TP 需要的维度信息。

5.6 ZeRO-Offload 与 ZeRO-Infinity

切分终究受限于「总显存 = N × 单卡容量」。当卡不够多时，还有一条路：**把状态挪到 GPU 外面**。

方案	挪什么	适用	代价
ZeRO-Offload	优化器状态 + 梯度计算 → CPU 内存，GPU 只做前向/反向	少卡（1~4 卡）训大模型	CPU-GPU 传输带宽成瓶颈（PCIe 4.0 x16 约 32 GB/s 量级）
ZeRO-Infinity	进一步用 NVMe SSD	万亿参数在有限 GPU 上	需要分块预取（prefetch）+ 计算/I/O 重叠

```
ZeRO-Offload 的数据流：
  GPU：前向 → 反向（得梯度）
        │ 梯度搬到 CPU（PCIe）
  CPU：Adam 更新（FP32，无 Tensor Core，慢但显存便宜）
        │ 更新后的参数搬回 GPU
  GPU：下一次前向
```

关键判断：Offload 的本质是「用 PCIe 带宽换显存」。如果模型的计算强度高（大矩阵乘），CPU 更新那点时间能被 GPU 计算掩盖；如果模型小、层多、通信频繁，PCIe 就会成为主瓶颈，吞吐可能掉一个数量级【推断】。

5.7 选型：一张表定方案

你的情况	选择	理由
单卡装得下完整状态	DDP	通信最省（2Ψ），别为了「显得高级」上 FSDP 白付 50% 通信
单卡差一点	FSDP `SHARD_GRAD_OP` / ZeRO-2	通信较省，省掉 75% 的优化器冗余
单卡差很多	FSDP `FULL_SHARD` / ZeRO-3	显存最省（16Ψ/N）
多机 + 大模型	FSDP `HYBRID_SHARD`	避免在慢速跨机链路上跑高频 AllGather

你的情况	选择	理由
卡少（1~4）+ 模型大	ZeRO-Offload	用 PCIe 换显存
单层都装不下	叠加 TP/PP	数据并行这条维度到头了（第 6、7 章）

> FSDP vs DeepSpeed 的选择：PyTorch 生态内优先 FSDP（无需额外依赖、与 `torch.compile`/DTensor 协同好）；需要 Offload/Infinity、或已有 DeepSpeed 配置与 MoE 支持时用 DeepSpeed。ZeRO 是**算法/论文概念**，FSDP 与 DeepSpeed 是两套**实现**——`FULL_SHARD ≈ ZeRO-3`、`SHARD_GRAD_OP ≈ ZeRO-2`。

5.8 本章小结与自检

1. DDP 的 `(N-1)/N` 存储是纯冗余；ZeRO 用「切分-聚合」消除它。
2. 三阶段公式必须背下来：`4Ψ+12Ψ/N` / `2Ψ+14Ψ/N` / `16Ψ/N`。
3. 通信账：DDP `2Ψ`、ZeRO-3/FSDP `3Ψ`（多出的 Ψ 是前向那次 AllGather）。
4. FSDP 的四种策略是「显存 vs 通信」的旋钮；多机首选 `HYBRID_SHARD`。
5. Offload 用 PCIe 带宽换显存，Infinity 再往外扩到 NVMe。

自检题：

1. 推导 ZeRO-1/2/3 的每卡显存公式，并解释每一项的物理含义。
2. 为什么 ZeRO-2 的通信量「看起来」和 DDP 一样？它到底省了什么、补了什么？
3. FSDP 每步 3Ψ，这多出来的 Ψ 花在哪？什么条件下它「不亏」？
4. 8 卡 70B 模型：DDP / ZeRO-2 / ZeRO-3 各自单卡静态显存是多少？哪个可行？

第 6 章 张量并行与序列并行：切一层矩阵

> 对应 AIInfraGuide 模块三 第 6 章（`第6章-张量并行与序列并行` 扉页）。

6.1 为什么 DP/FSDP 到头了

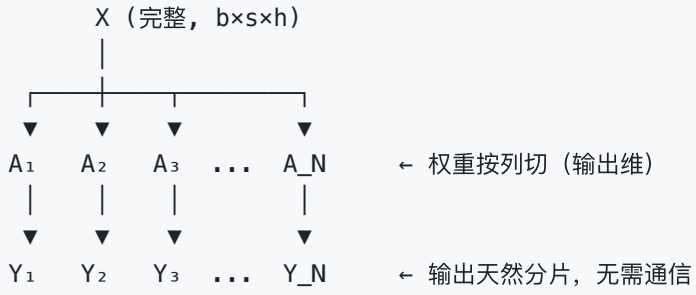
DP/FSDP 切的是「训练状态的冗余副本」，但如果**单层的前向计算本身**就超出单卡（巨大的 FFN 中间维度、巨大的 vocab 投影），DP 无能为力——因为每一层都得在每张卡上完整算一遍。

TP 的定位：把**单个矩阵乘法** `Y = X·A` 拆到多卡，每卡只存 `1/N` 的权重和对应的部分激活。

> 代价：TP 每层都要通信，对延迟极其敏感 → **几乎总是限制在 NVLink 互联的单机 8 卡内**。

6.2 Column Parallel Linear：按列切

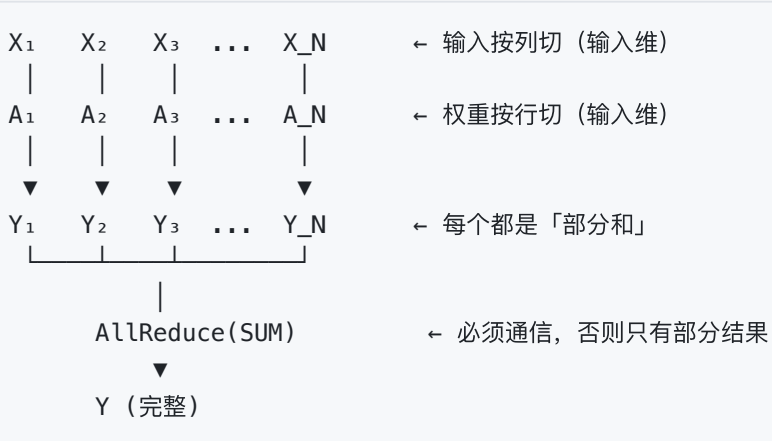
```
A 按列切:  A = [A₁ | A₂ | ... | A_N]      每卡持 Aᵢ（形状 h × (h_out/N)）
计算:      Yᵢ = X · Aᵢ                     每卡用「完整输入 X」乘自己的列分片
输出:      Y = [Y₁ | Y₂ | ... | Y_N]       各卡持 Y 的不同列分片（沿输出维切分）
通信:      前向不需要（各卡本就有完整 X）；反向需要 AllReduce（对 X 的梯度要求和）
```



应用位置：**FFN 的第一个线性层**（切中间维度）、**Attention 的 QKV 投影**（多头天然可切，每卡负责部分 Head）。

6.3 Row Parallel Linear：按行切

```
A 按行切:  A = [A₁; A₂; ...; A_N]          每卡持 Aᵢ（形状 (h_in/N) × h_out）
输入要求:  X 也要按列切为 Xᵢ（正好接 Column Parallel 的输出）
计算:      Yᵢ = Xᵢ · Aᵢ                    每卡算一个「部分和」
通信:      各卡 Yᵢ 通过 AllReduce(SUM) 得到完整 Y
```



应用位置：**FFN 的第二个线性层**（合并中间维度）、**Attention 的输出投影**。

6.4 Transformer Block 的 TP 切分：通信插入位置推导

这是本章最该「推」而不是「背」的部分。规则只有一条：**Column 后面接 Row，中间不用通信；一个 Column–Row 对，出口需要一次 AllReduce。**

```
输入 X (b,s,h)  ── 每个 rank 都持有完整副本（SP 之前）
  │
  ├─ LayerNorm ──────────────────────────── 非 TP 区（每卡完整）
  │
  ├─ QKV 投影  [Column Parallel]  ────────── 输出按 head 切
  │     │  每卡: Q_i, K_i, V_i（部分 head）
  ├─ Attention（各卡只算自己的 head）──────── 无通信
  │     │
  ├─ 输出投影  [Row Parallel]  ──────────── ▶ AllReduce #1  (f 前向)
  │
  ├─ LayerNorm ──────────────────────────── 非 TP 区
  │
  ├─ FFN 第一层 [Column Parallel] ────────── 中间维 4h 切成 4h/N
  │     │  激活函数（GELU）逐元素，无需通信
  ├─ FFN 第二层 [Row Parallel]  ──────────── ▶ AllReduce #2  (g 前向)
  │
  └─ 输出（每卡完整）
```

结论：每个 Transformer Block 前向需要 2 次 AllReduce（Attention 输出 + FFN 输出），反向对称地再 2 次。

位置	切法	为什么这样配
QKV 投影	Column	多头天然可分，每卡算一部分 head
Attention 输出投影	Row	把各 head 的部分和合并回完整 h 维
FFN 第一层	Column	切 4h 的中间维（这是 TP 省显存的主力）
FFN 第二层	Row	把 4h/N 的部分和合并回 h

6.4.1 通信量

原文给出的口径：**每层前向 2 次 AllReduce，每次传输 `2bsh` 数据**。

```
每层每卡通信量 ≈ 2 × 2bsh = 4bsh          （前向）
反向对称，再 4bsh
```

> 口径说明【推断】：AllReduce 的张量本身是 `b×s×h` 个元素；写成 `2bsh` 是把「发送 + 接收」两个方向合起来计。不同资料口径不同，**比较时务必用同一口径**——这里沿用原文表述，避免在数字上打对不上。

量级感：7B 模型、b=1、s=2048、h=4096、32 层、TP=8。每层每卡前向 `4×1×2048×4096×2 B ≈ 67 MB`（BF16），32 层约 2.1 GB/步【量级】——和梯度同步的 14 GB 比不算大，但它**每层都要发生一次**，延迟敏感度完全不同。这就是 TP 必须放机内的原因。

6.5 序列并行 SP：补上 TP 的漏洞

TP 只切了 Linear，**LayerNorm 和 Dropout 不参与切分**——它们的输入/输出在每卡上是完整的。这部分激活值冗余，在长序列下很可观。

SP 的做法：在非 TP 区域（LayerNorm、Dropout、残差），**沿序列维切分激活值**。

```
TP only:    LN 输入每卡完整 (b,s,h)          → 激活冗余 N 倍
TP + SP:    LN 输入每卡 (b, s/N, h)          → 激活降低 N 倍
```

关键机制是**通信原语的替换**：

位置	只有 TP	TP + SP
Row Parallel 出口（进非 TP 区）	AllReduce	**ReduceScatter**
Column Parallel 入口（回 TP 区）	（无需）	**AllGather**

```
AllReduce  =  ReduceScatter  +  AllGather
              └ 进非 TP 区时做     └ 回 TP 区时做
```

总通信量不变（还是 2Ψ 那一套），但激活显存从完整 `bsh` 降到 `bsh/N`。原文给了一句精确的总结：SP 让「Non-TP 区域的激活显存从完整 bsh 降至 bsh/N，总激活显存接近线性缩放」。

> 名字上要分清三件事（第 9 章会再强调）： – **TP**：切 Linear 的权重矩阵； – **SP（Megatron 序列并行）**：切 TP 覆盖不到的非 Attention 区域激活（LN/Dropout），切的是**序列维**； – **CP（上下文并行）**：切 **Attention 计算本身**（第 9 章）。

6.6 GQA / MQA 下的 TP：KV Head 不够切怎么办

GQA 里 KV Head 数可能小于 TP 度（如 KV Head=4、TP=8），此时无法「每卡一个 KV Head」。

策略	做法	代价
复制 KV Head	每个 TP rank 复制一份完整 KV Head	KV 参数切分不完全均匀，显存省得少一点
分组共享	相邻 rank 共享同一 KV Head	需要额外的通信/索引逻辑

> 这是工程实现细节（Megatron 里由 `num_query_groups` 与 TP 度的整除关系决定走哪条路径）。**推断**：当 `num_kv_heads < tp_size` 时，实际做法是把 KV Head 复制 tp_size/num_kv_heads 份——这不会破坏计算正确性，只是让 KV 投影的权重分片出现重复。

6.7 手写一个最小 TP（Column + Row）

```
import torch, torch.nn as nn, torch.distributed as dist

class ColumnParallelLinear(nn.Module):
    """Y = X @ A，A 按输出维切分；每卡算 Y 的一部分列"""
    def __init__(self, in_features, out_features, tp_rank, tp_size):
        super().__init__()
        assert out_features % tp_size == 0
        self.out_per_rank = out_features // tp_size
        self.weight = nn.Parameter(torch.empty(self.out_per_rank, in_features))
        nn.init.kaiming_uniform_(self.weight)

    def forward(self, x):                       # x: (..., in_features) 完整
        return x @ self.weight.t()              # (..., out_per_rank)  — 无需通信

class RowParallelLinear(nn.Module):
    """Y = X @ A，A 按输入维切分；每卡算部分和，最后 AllReduce"""
    def __init__(self, in_features, out_features, tp_rank, tp_size, group=None):
        super().__init__()
        assert in_features % tp_size == 0
        self.in_per_rank = in_features // tp_size
        self.group = group
        self.weight = nn.Parameter(torch.empty(out_features, self.in_per_rank))
        nn.init.kaiming_uniform_(self.weight)

    def forward(self, x):                       # x: (..., in_per_rank) 已是分片
        partial = x @ self.weight.t()           # (..., out_features) 部分和
        dist.all_reduce(partial, op=dist.ReduceOp.SUM, group=self.group)
        return partial
```

验证方法（这是判断实现对不对的唯一标准）：单卡跑一个 2 层 MLP 得到 `y_ref`；再用 2 卡 TP 跑同样的权重切分，比较 `allclose(y_ref, y_tp, atol=1e-5)`。BF16 下 `atol` 放宽到 `1e-2`。

6.8 本章小结与自检

1. TP 切的是「单层矩阵」，解决「单层太大」，代价是每层都要通信。
2. Column 切输出维（出口无需通信）、Row 切输入维（出口必须 AllReduce）；**Column→Row 配对**是 Transformer 的标准布局。
3. 每个 Transformer Block 前向 2 次 AllReduce（Attention 出口 + FFN 出口），反向对称。

4. SP 补的是 LayerNorm/Dropout 区域的激活冗余：AllReduce 拆成 ReduceScatter + AllGather，**通信量不变、激活降 N 倍**。
5. GQA 下 KV Head 少于 TP 度时需要复制或分组，会影响切分均匀性。

自检题：

1. 画出 TP=4 时一个 Transformer Block 的切分图，标出每次通信的位置和原语。
2. 为什么 Column Parallel 的输出可以直接喂给 Row Parallel 而中间不用通信?
3. SP 之后总通信量为什么没变？那它的收益到底在哪?
4. KV Head=8、TP=16 时会发生什么？怎么处理?

第 7 章 流水线并行：Bubble 从 1 到 1/(M·v)

> 对应 AllnfraGuide 模块三 第 7 章（`第7章-流水线并行` 扉页）+【补充】GPipe / Megatron-LM 1F1B 论文。

7.1 基本思想与第一个代价

把模型的 L 层切成 P 段，每段放一台/一组机器，激活值像流水线一样在段间传递：

```
Stage0 [层 1..L/P] → Stage1 → Stage2 → Stage3 → loss
        每个 Stage 负责 L/P 层，段间只传 b×s×h 的激活值
```

通信量极小（只在段边界传一次激活），所以 PP 是**跨机扩展的主力**。但它引入了一个新代价：

```
朴素 PP（一个 micro-batch 串行经过所有 Stage）：

时间 →
S0  [F]
S1     [F]
S2        [F]
S3           [F]
             ↑ 同一时刻只有 1 个 Stage 在干活，其余全在等
```

Bubble 比例（朴素 PP）= (P−1)/P。P=4 时 75% 的时间是浪费的——4 张卡只有 1 张在算。

7.2 三代调度：同一个 Bubble 公式，不同的显存

7.2.1 GPipe：切 micro-batch

把 mini-batch 切成 M 个 micro-batch，依次注入流水线：

```
GPipe (M=4, P=4) :
S0  F1 F2 F3 F4 -  -  -  B4 B3 B2 B1
S1  -  F1 F2 F3 F4 -  -  -  B4 B3 B2 B1
S2  -  -  F1 F2 F3 F4 -  -  -  B4 B3 B2
S3  -  -  -  F1 F2 F3 F4 -  -  -  B4 B3 B2 B1
                ↑ 所有 F 做完，才开始 B
```

项	值
Bubble 比例	`(P-1) / (M + P - 1)`
峰值激活显存	**M 份**（所有 micro-batch 的激活都要留着等反向）
问题	M 越大 Bubble 越小，但显存线性爆炸

7.2.2 1F1B：省显存的关键一步

核心思想：**交替执行前向和反向，尽早释放激活值**。分三段：

```
1F1B (M=8, P=4) :
        ├── Warmup ──┤├──── Steady (1F1B) ────┤├ Cooldown ─┤
S0  F1 F2 F3 F4  B1 F5 B2 F6 B3 F7 B4 F8 B5 B6 B7 B8
S1     F1 F2 F3  B1 F4 B2 F5 B3 F6 B4 F7 B5 B6 B7 B8
...
```

项	值
Bubble 比例	`(P-1) / (M + P - 1)` —— **与 GPipe 相同**
峰值激活显存	**P 份**（同一时刻最多 P 个 micro-batch 的激活存活）
地位	**Megatron-LM 默认调度**

> 这是本模块最漂亮的一个结论：**1F1B 不改善 Bubble，只改善显存**。Bubble 一样，但峰值激活从 M 份降到 P 份——这让你可以把 M 开得更大，从而间接把 Bubble 压得更小。工程上「同一个公式，两个不同的自由度」。

7.2.3 Interleaved 1F1B：虚拟 Stage

让每个设备不只负责连续的 L/P 层，而是负责 **v 段非连续**的层（虚拟 Stage），每段 `L/(P·v)` 层：

```
Interleaved（P=4, v=2）：
设备0 负责 [层 1..L/8] 和 [层 L/2+1..5L/8]
相当于把「流水线段数」从 P 变成 P·v，M 的有效值从 M 变成 M·v

Bubble 比例 = (P−1) / (M·v + P − 1)
```

项	值
Bubble 比例	`(P−1)/(M·v + P − 1)`，等效把 M 扩大 v 倍
峰值激活	P 份（不变）
代价	通信频率增加 **v 倍**（每个虚拟 Stage 之间都要传激活）
适用	机间带宽充足时，用额外通信换更低 Bubble

7.3 四种调度横向对比

调度策略	Bubble 比例	峰值激活显存	通信频率	典型 P=4, M=8
朴素 PP	(P−1)/P	1 份	低	75%
GPipe	(P−1)/(M+P−1)	M 份	低	27.3%
1F1B	(P−1)/(M+P−1)	**P 份**	低	27.3%
Interleaved 1F1B（v=2）	(P−1)/(M·v+P−1)	P 份	高（v 倍）	15.8%

【补充】算一下就能看出为什么大家最后都选 1F1B：P=4、M=8 时 GPipe 和 1F1B 的 Bubble 都是 3/11 ≈ 27.3%，但 1F1B 的峰值激活只有 P=4 份而不是 M=8 份。**同样的 Bubble，一半的显存**——这才是 1F1B 被 Megatron 定为默认的原因。

7.3.1 Bubble 的绝对时间

Bubble 比例是相对值，落到绝对时间上：

```
T_bubble ≈ (P−1) × t_microbatch_forward          （每个 Bubble 槽 = 一次 micro-batch 的 Stage 计
T_total  ≈ M × t_microbatch × (1 + Bubble比例)
```

推论：**Stage 越少（P 小）、micro-batch 越多（M 大），Bubble 越不疼**。所以 PP 的并行度通常不会开得很大（常见 2~16），并且总是配合较大的 M。原文给的实战练习是「4 Stage、8 Micro-batch 画 GPipe 与 1F1B 的时间线」——建议真的画一遍，画完这两个公式就再也不会记错。

7.4 PP 的四个工程挑战

挑战	问题	做法
负载均衡	不同层计算量不同（Embedding 层、MoE 层、Loss 层）	按「实测 FLOPs」而非「层数」分配；Embedding/Loss 单独放首尾 Stage
Embedding / Loss 位置	Embedding 在模型最前、Loss 在最后	通常放第一个和最后一个 Stage，注意这会打破均匀切分
段间激活传输	每次传 `b×s×h`	需要 IB 带宽支撑；可与计算重叠（`batch_isend_irecv`）
与重计算配合	反向要重算激活	PP 天然适合 recomputation：每个 Stage 只存自己的激活（第 8 章）

Stage 间传多少：`b × s × h × dtype_bytes`。b=1、s=2048、h=4096、BF16 → 约 16 MB/次。相比梯度的 GB 级，这个量很小，所以 PP 能跨机。

7.5 手写一个 PP 调度模拟器（理解 Bubble 的最快方式）

不用真的多卡，用「时间步矩阵」就能把三种调度画出来：

```
def simulate_gpipe(P, M):
    """GPipe：先做完全部 F，再做完全部 B。返回 timeline[stage][t]"""
    T = 2 * (M + P - 1)
    tl = [[" ."] * T for _ in range(P)]
    for i in range(M):                          # 前向：F_i 在 stage s 的时隙 = i + s
        for s in range(P):
            tl[s][i + s] = f"F{i}"
    off = M + P - 1
    for i in reversed(range(M)):                # 反向：倒序回传
        for s in range(P):
            tl[s][off + (M - 1 - i) + (P - 1 - s)] = f"B{i}"
    return tl

def bubble(tl):
    tot = sum(len(r) for r in tl)
    return sum(1 for r in tl for c in r if c == " .") / tot

def bubble_ratio(P, M, v=1):                    # 教材口径：一步 = 一次 F + 一次 B
    return (P - 1) / (M * v + P - 1)

for P, M in [(4, 8), (4, 32), (8, 16), (16, 32)]:
    print(f"P={P:2d} M={M:3d} → 实测 {bubble(simulate_gpipe(P, M)):6.2%}"
          f"  公式 {bubble_ratio(P, M):6.2%}")

for row in simulate_gpipe(4, 6):                # 打印时间线，应看到标准梯形
    print(" ".join(f"{c:>3}" for c in row))
```

为什么只模拟 GPipe：GPipe 的「全部 F → 全部 B」在时隙上是**唯一确定**的，模拟值与公式 `(P-1)/(M+P-1)` **精确相等**（可以自己跑一遍验证，P=4/M=8 应得 27.27%）。1F1B 与 Interleaved 的时隙分配依赖具体实现（warmup 长度、micro-batch 排队策略），照搬一个「看起来对」的模拟器只会得到似是而非的数——**它们的 Bubble 用公式算，不要用模拟器猜**。

这一步别跳过：把 `simulate_gpipe` 的输出画出来，你会看到标准梯形——梯形左上和右下两块空白就是 Bubble，剩下的宽度是有效流水。Bubble 公式只有自己数过一遍才不会记混。

7.6 本章小结与自检

1. PP 切层、通信量小（只传 `b·s·h` 的激活）、可跨机，代价是 Bubble。
2. Bubble 公式：朴素 `(P-1)/P`；GPipe 与 1F1B 都是 `(P-1)/(M+P-1)`；Interleaved 是 `(P-1)/(M·v+P-1)`。
3. **1F1B 不改善 Bubble，只把峰值激活从 M 份降到 P 份**——这是它成为默认调度的原因。
4. Interleaved 用 v 倍通信换 Bubble 下降，只在机间带宽充足时划算。
5. 工程上要处理：负载均衡、Embedding/Loss 位置、段间传输重叠、与重计算配合。

自检题：

1. P=8、M=16 时 1F1B 的 Bubble 是多少？要压到 10% 以下需要多大的 M？

2. 为什么 GPipe「M 越大越好」在实践中行不通?

3. Interleaved 的 v 能无限增大吗?瓶颈在哪?

4. PP 与 TP 都切模型,为什么 PP 能跨机而 TP 不能?用通信量解释。

第 8 章 混合精度与显存优化:单卡内部的最后一公里

> 对应 AIInfraGuide 模块三 第 8 章(`第8章-混合精度与显存优化` 扉页)。

并行解决了「切到多卡」,但单卡内部还有大量显存可以省。本章三类技术与并行维度**正交**,几乎所有训练都会用到:混合精度、梯度累积、Activation Checkpointing。

8.1 四种数值格式

类型	位宽	指数位	尾数位	数值范围	相对精度	适用
FP32	32	8	23	大	~1e-7	Master weight、优化器
TF32	19	8	10	同 FP32	~1e-3	Ampere+ 的矩阵乘加速
FP16	16	5	10	**小,易溢出**	~1e-3	旧方案
BF16	16	8	7	**与 FP32 相同**	~4e-3	大模型训练首选
FP8 (E4M3)	8	4	3	很小	~1e-1	H100+ 前向计算

为什么混合精度能省显存又提速:位宽减半 → 显存/带宽减半;同时 Tensor Core 在低精度下算力更高(H100 上 BF16 约为 FP32 的十余倍,FP8 再翻倍【量级】)。

8.1.1 BF16 混合精度的完整流程

```
前向:  参数(BF16) → 计算(BF16) → 激活(BF16)
反向:  梯度(BF16)
更新:  梯度 → 转 FP32 → 加到 FP32 master weights 上做 Adam 更新
       → 结果转回 BF16 → 供下一次前向
```

这就是第 3 章那 12Ψ 的来源:**FP32 主权重不是可选优化,是混合精度训练能收敛的前提**。

8.1.2 FP16 vs BF16:为什么大模型选 BF16

维度	FP16	BF16
指数位	5(范围 ±65504)	8(范围同 FP32)

维度	FP16	BF16
尾数位	10（精度高）	7（精度低）
溢出风险	**高**（梯度稍大就 inf）	低
需要 Loss Scaling	**必须**	通常不需要
大模型实践	早期（GPT-2 时代）	当前默认

一句话：**BF16 用精度换稳定**。大模型的激活和梯度动态范围极宽，FP16 那 5 位指数根本兜不住，于是要把精力花在 loss scaling 上；BF16 直接把这个麻烦消掉了。

8.1.3 Loss Scaling：FP16 的必要补丁

FP16 的问题：梯度值常小到 2^{-14}（最小正规数）乃至 2^{-24}（最小次正规数）以下，直接算会下溢成 0。解法是**先把 loss 放大 2^k 倍**，反向得到的梯度也同比例放大，更新前再除回来：

```
loss_scaled = loss × S
loss_scaled.backward()          # 梯度被放大 S 倍，避开下溢
grad = grad / S                 # 更新前还原
```

方式	做法	优缺点
静态	手工设 S（如 2^{12}）	简单；S 太大会 inf，太小会下溢
动态	每隔 N 步检查是否有 inf/nan：有则减半 S 并跳过该步；连续 M 步正常则加倍 S	PyTorch `GradScaler` 的默认行为，**推荐**

```
scaler = torch.cuda.amp.GradScaler()
for xb, yb in loader:
    opt.zero_grad()
    with torch.cuda.amp.autocast(dtype=torch.float16):
        loss = criterion(model(xb), yb)
    scaler.scale(loss).backward()      # 放大后再反向
    scaler.step(opt)                   # 内部会 unscale + 检查 inf
    scaler.update()                    # 调整 S
```

> BF16 通常不需要 scaler（范围够大），直接用 `autocast(dtype=torch.bfloat16)` 即可。**但注意**：某些实现里 BF16 也会配 scaler 做数值保护，不是错，只是多余。

8.1.4 FP8 训练：能省但要小心

H100 及之后的 Transformer Engine 支持 FP8。典型配置是**前向用 FP8 + per-tensor 动态缩放，反向梯度用 BF16**：

项	做法
缩放	per-tensor scaling（每个张量一个 scale），amax 历史滚动更新
哪些层用	通常是 Linear（GEMM），LayerNorm/Softmax 仍用高精度
风险	精度损失集中在小梯度、大动态范围处；需要 `fp8_format`（E4M3 前向 / E5M2 反向）与 recipe 调参
收益	显存与带宽再减半，GEMM 吞吐再翻倍【量级】

⚠️ 实践经验：FP8 的收益在**大 GEMM、长序列**上明显，在小模型上经常被缩放开销和精度回退抵消。别默认开。

8.2 梯度累积：小显存模拟大 batch

目的：在有限显存下模拟更大的 effective batch。

```
机制：连续 K 步「前向 + 反向」累积梯度，每 K 步才做一次 optimizer.step()

Effective Batch Size = 单卡 batch × world_size × accumulation_steps
```

```
for i, (xb, yb) in enumerate(loader):
    # 中间步不同步梯度（省通信！）
    ctx = model.no_sync() if (i + 1) % K != 0 else contextlib.nullcontext()
    with ctx:
        loss = criterion(model(xb), yb) / K      # ← 关键：除以 K
        loss.backward()
    if (i + 1) % K == 0:
        opt.step(); opt.zero_grad()
```

要点	说明
Loss 要除以 K	否则梯度是 K 倍，等价于 lr 放大 K 倍（与第 4 章「不能求和」同源）
`no_sync()`	DDP 下中间步跳过 AllReduce，**通信量降到 1/K**
与 PP 的关系	PP 的 micro-batch 本身就是一种「累积」，梯度累积进一步放大
与 BN 的关系	梯度累积会改变 BN 的统计口径（每个 micro-batch 单独算），大 batch 语义有偏差

> 【补充】`no_sync()` 是 DDP 上最容易被忽略的性能开关：不做梯度累积时它没用；一做累积，通信量直接降到 1/K，是「免费」的加速。

8.3 Activation Checkpointing：用计算换显存

前向保存的激活值随层数和 batch 线性增长，大模型训练中**激活显存可能超过参数显存**。

核心思想：只保存部分层的激活（checkpoint），其余层反向时**重新前向计算**。

```
无重计算:   存所有层的激活              显存 O(L)
Full CKPT:  只存每个 Block 的输入       显存 O(L) → O(√L)   计算 +约 33%
Selective:  只重算「计算量小但激活大」的算子（如 attention 的 softmax 输出）
            保留「计算量大」的激活（如 GEMM 的输出）
```

方案	存什么	显存	额外计算	适用
无	全部激活	O(L)	0	小模型 / 短序列
Full	每个 Block 的输入	O(√L)（每 √L 层存一个）	约 +33% 前向	大模型默认
Selective	只存 GEMM 输出等大算子结果，重算便宜的	介于两者之间	+约 5%~10%【量级】	想要「几乎免费」的省显存

```
from torch.utils.checkpoint import checkpoint

def forward(self, x):
    for layer in self.layers:
        if self.gradient_checkpointing and self.training:
            x = checkpoint(layer, x, use_reentrant=False)
        else:
            x = layer(x)
    return x
```

> `use_reentrant=False` 是新版推荐值：支持 `autograd.grad`、支持不进反传的输入、与 FSDP/TP 组合更安全。老代码里的 `use_reentrant=True` 有已知限制。

8.4 显存优化全景：一张表收束

把全模块的「省显存」手段按机制归类：

类别	手段	作用于	章节
切分（多卡）	ZeRO/FSDP	优化器状态、梯度、参数	第 5 章
切分（多卡）	TP	单层权重	第 6 章
切分（多卡）	PP	层	第 7 章

类别	手段	作用于	章节
切分（多卡）	SP / CP	激活 / Attention	第 6、9 章
降精度	BF16 / FP8	参数、梯度、激活	本章
时间换空间	梯度累积	激活（等效小 batch）	本章
时间换空间	Activation CKPT	激活	本章
卸载	ZeRO-Offload / Infinity	优化器状态、参数 → CPU/NVMe	第 5 章

对应的「显存四大组成 × 谁能动它」：

组成	大小	能被谁压
参数	2Ψ	降精度、ZeRO-3、TP
梯度	2Ψ	ZeRO-2/3、梯度累积（间接）
优化器状态	12Ψ	ZeRO-1/2/3、Offload、换优化器
激活	∝ b·s·h·L	小 batch、CKPT、SP、CP、PP

8.5 本章小结与自检

1. BF16 用精度换稳定，是大模型默认；FP16 必须配 loss scaling。
2. FP8 收益在大 GEMM，缩放与精度风险要单独调。
3. 梯度累积的 effective batch = `单卡 batch × world_size × K`；loss 要除以 K，DDP 要配 `no_sync()`。
4. Full CKPT 把激活从 O(L) 降到 O(√L)，代价约 +33% 前向；Selective 更便宜。
5. 省显存手段分四类：切分、降精度、时间换空间、卸载——**先判断你的瓶颈是哪一项**，再选手段。

自检题：

1. 为什么 Adam 的 master weights 必须是 FP32，而前向可以用 BF16?
2. 梯度累积 K=4 时，DDP 的通信量变成多少？不调 `no_sync()` 会怎样?
3. Full CKPT 的「+33%」是怎么来的？（提示：一次前向 + 一次重算 ≈ 4/3）
4. 你的训练 OOM，但参数只有 20 GB、卡有 80 GB。列出至少 4 个可能吃掉 60 GB 的东西和对应手段。

第 9 章 长序列与上下文并行：切 Attention 本身

> 对应 AIInfraGuide 模块三 第 9 章（`第9章-长序列训练与上下文并行` 扉页）+【补充】FlashAttention、Ring Attention、DeepSpeed-Ulysses 论文。

9.1 序列长度墙：为什么别的并行都不管用

序列长度从 2K 涨到 128K 甚至 1M 后，两件事同时恶化：

```
Attention 计算量:  O(s²·h)        s 翻倍 → 计算翻 4 倍
Attention 显存:    O(s²·heads)    s 翻倍 → score 矩阵翻 4 倍（若无 FlashAttention）
KV Cache / 激活:   O(s·h)         s 线性增长
```

为什么 DP / TP / PP 都解决不了「单条序列太长」？

策略	为什么不行
DP	每卡都持完整序列 → 一点没省
PP	层被切了，但**每一层内的单条序列还是完整的** → 该层的 s^2 激活照样爆
TP	切的是 h 维（权重矩阵），s 维没动 → 序列越长，每条序列的激活仍在一张卡上
SP（第 6 章）	切的是 LN/Dropout 区域，**Attention 计算本身仍要完整序列**

结论：必须引入一个新维度，**沿序列维切分 Attention 计算**——上下文并行（Context Parallelism, CP）。

> 名字辨析（最容易混的三件事）：
> - **SP（Megatron Sequence Parallel）**：切非 Attention 区域的激活，随 TP 一起开，目标是省 LN/Dropout 的冗余。
> - **CP（Context Parallel）**：切 Attention 计算本身，目标是让 128K 序列的单层能放下。
> - **Ulysses / Ring Attention**：CP 的两种**具体实现**，切分维度不同。

9.2 FlashAttention：CP 的前置知识

FlashAttention 用「分块 + online softmax」把 Attention 的**显存**从 $O(s^2)$ 降到 $O(s)$（计算量仍是 $O(s^2)$，但不再materialize 完整 score 矩阵）：

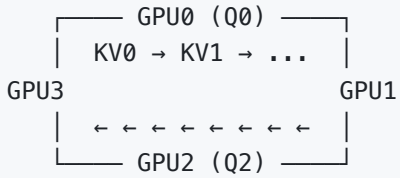
```
标准 Attention:  S = QKᵀ（s×s 显存）→ softmax(S) → O = S·V
FlashAttention:  按块遍历 K/V，维护 running max 与 running sum，边算边合并
                 → 只需 O(s) 的显存，且访存次数大幅减少
```

关键性质（CP 能成立的前提）：分块 attention 对「K/V 块的处理顺序**不敏感**」——因为 online softmax 的合并是可交换的。这就是为什么 Ring Attention 可以把 KV 块沿环传递、每卡用不同顺序处理，最后 `lse`（log-sum-exp）合并仍得到精确结果。

9.3 Ring Attention：环传 KV

沿序列维把 Q/K/V 切成 N 份，每卡持一份：

```
每卡持 Q_i, K_i, V_i（第 i 段序列）
for step in range(N):
    计算 Attention(Q_i, K_{i+step}, V_{i+step}) → 用 online softmax 合并进 running 结果
    把（K, V）发给下一个邻居，同时收上一个邻居的（K, V）   ← P2P，可与计算重叠
```



项	说明
通信原语	P2P 环传 KV（不是 All-to-All）
通信量	每卡传 `(N-1)/N × 2·s·h` 量级的 KV【量级】
优势	序列长度可随卡数**线性扩展**，且通信能与计算重叠（这是它的设计核心）
问题	**因果掩码下负载不均**：GPU_i 只需算前 i 段 KV，后面的卡早早算完在等
优化	**Zigzag / Striped** 重排：把序列块按「之字形」分配（GPU0 拿头尾两块），使每卡的因果工作量接近相等

因果掩码的负载不均（N=4，每个三角块 = 一份计算量）：



9.4 DeepSpeed-Ulysses：Head → 序列的 All-to-All 换轴

Ulysses 的思路完全不同：**先沿 Head 维切，再用 All-to-All 换成沿序列维切**。

```
输入: 每卡持「全部序列 + 1/N 的 Head」        (切 Head)
  │
  ├─ All-to-All  #1 (重排: Head 维 → 序列维)
  │
Attention: 每卡持「1/N 序列 + 全部 Head」      (切序列)
  │
  ├─ All-to-All  #2 (重排: 序列维 → Head 维)
  │
输出: 回到「全部序列 + 1/N 的 Head」
```

项	说明
通信原语	All-to-All ×2（进、出各一次）
通信量	与**序列长度无关**，与 `s·h/N` 成正比；一次 All-to-All 的量级是 `s·h/N` 【量级】
硬限制	**Head 数必须 ≥ CP 度**（否则没法按 Head 切）
优势	通信量不随 s 增长；实现简单，NCCL 的 All-to-All 优化成熟
劣势	Head 少的模型（如 MQA / 小模型）不能用；两次 All-to-All 有固定延迟

9.5 Megatron Context Parallel

Megatron 的 CP 走的是「沿序列切 + 配合 FlashAttention 分块」的路线，通信以 P2P/AllGather 为主，深度绑定 Megatron 的 TP/SP 体系：

项	说明
切分维度	序列
通信	P2P（KV 交换）/ AllGather，与 TP/SP 的通信域协同
优势	能与 TP + SP + PP 同时开，是 Megatron 生态里搭 4D 并行的标准件
劣势	与 Megatron 代码强绑定，独立使用成本高

9.6 三种方案对比

方案	切分维度	通信原语	主要限制	适用场景
Ring Attention	序列	P2P 环传 KV	因果掩码负载不均（需 Zigzag）	超长序列、KV 大、Head 少
Ulysses	Head ⇄ 序列	All-to-All ×2	Head 数 ≥ CP 度	Head 多、想通信量与 s 解耦
Megatron CP	序列	P2P / AllGather	与 Megatron 绑定	Megatron 生态、要组 4D

选型直觉：

```
Head 数 ≥ CP 度?
├─ 是 → 优先 Ulysses（通信量与 s 无关，最省心）
└─ 否 → Ring Attention（不依赖 Head 数）+ Zigzag 解决负载不均
        └ 若已在 Megatron 生态 → 直接用 Megatron CP
```

> 【推断】实际生产中常见「混搭」：Ulysses 处理 Head 维、Ring 处理序列维（如 2D CP），以同时绕开「Head 数限制」和「通信量随 s 增长」。这类方案在超长上下文（512K+）的训练里出现得越来越多。

9.7 一个最小 Ring Attention 骨架

```python
def ring_attention_step(q, k_local, v_local, rank, world_size):
    """每卡持一段 Q/K/V；环传 KV，online softmax 合并。返回本卡的输出。"""
    out = torch.zeros_like(q)
    lse = torch.full(q.shape[:-1] + (1,), -float("inf"), device=q.device)

    k, v = k_local, v_local
    for step in range(world_size):
        # 1) 用当前 KV 块算局部 attention（FlashAttention 内核）
        block_out, block_lse = flash_attn_with_lse(q, k, v)
        # 2) online 合并：新的 lse 取 max，输出按 exp 权重加权
        new_lse = torch.logaddexp(lse, block_lse)
        out = out * torch.exp(lse - new_lse) + block_out * torch.exp(block_lse - new_lse)
        lse = new_lse
        # 3) 环传 KV（与下一轮计算重叠）
        k, v = ring_send_recv(k, v, rank, world_size)
    return out / torch.exp(lse)     # 归一化
```

验证方法：单卡跑完整 attention 得 `ref`；4 卡跑 ring 得 `out`，比较 `allclose(out, ref, atol=1e-2)`（BF16）。**如果只在 N=1 或 N=2 上验证，很多合并 bug 会被掩盖**——一定要在 N≥4 上验。

9.8 本章小结与自检

1. Attention 的 $O(s^2)$ 让 DP/TP/PP 全部失效，必须沿序列维切——这是 CP。
2. CP ≠ SP：SP 切非 Attention 区域（配合 TP），CP 切 Attention 计算本身。
3. Ring Attention：P2P 环传 KV，靠 FlashAttention 的块序无关性保证精确；因果掩码下要 Zigzag 重排。
4. Ulysses：Head ↔ 序列的两次 All-to-All，通信量与 s 解耦，但要求 Head 数 ≥ CP 度。
5. 三种方案不是互斥的，可按 Head 数与序列长度混搭。

自检题：

1. 为什么 PP 切了层，仍然救不了 128K 序列的激活？

2. Ring Attention 为什么能得到**精确**结果而不是近似？依赖 FlashAttention 的什么性质？
3. Ulysses 在 MQA（只有 1 个 KV Head）上能开吗？为什么？
4. 因果掩码下普通切分的负载比是多少（N=4、N=8）？Zigzag 把它压到多少？

第 10 章 MoE 并行：All-to-All 与负载均衡

> 对应 AIInfraGuide 模块三 第 10 章（`第10章-moe并行` 扉页）+【补充】DeepSeek-V3 技术报告、Mixtral、GShard/Switch Transformer。

10.1 稠密 FFN → 稀疏 MoE

```
稠密 FFN:   x → Linear(h, 4h) → GELU → Linear(4h, h) → y
            每个 token 都过同一个 FFN，计算量 = 参数量 × 常数

MoE FFN:    x → Router（打分）→ Top-K 个 Expert → 加权求和 → y
            E 个 Expert（每个都是一整套 FFN），每 token 只用其中 K 个
```

项	稠密	MoE
参数量	Ψ	≈ Ψ × E（Expert 占大头）
单 token 计算量	正比于 Ψ	正比于 Ψ × K/E（**稀疏**）
显存	正比于 Ψ	正比于 Ψ × E（**照样要存全部 Expert**）
并行新维度	—	Expert Parallelism（EP）

这就是 MoE 的核心取舍：参数量（显存）随 E 涨，计算量只随 K 涨。用显存换「单位计算的模型容量」。

10.1.1 Router 与容量因子

```
Router:  gate = softmax(x · W_g)        # W_g: h × E
         topk_idx, topk_w = topk(gate, K)
         y = Σ_{i∈topk}  topk_w[i] · Expert_i(x)
```

概念	含义	工程影响
Top-K	每 token 激活几个 Expert（常见 K=2）	K 越大计算越多、路由越平滑
Capacity Factor（CF）	每个 Expert 最多收 `CF × (tokens/E)` 个 token	CF 太小 → 溢出丢 token（精度损失）；CF 太大 → 显存/计算浪费

概念	含义	工程影响
Token Drop	超容量的 token 被丢弃（或 pad 后丢弃）	训练时要监控 drop 率

10.2 Expert Parallelism

核心思想：**不同 Expert 放在不同 GPU 上**，每卡只存 `E/N` 个 Expert。

对比	TP	EP
切什么	单个权重矩阵（按行/列）	「哪些 Expert 在哪张卡」
通信	每层 AllReduce	每层 All-to-All ×2
卡内要求	高频、必须机内	带宽敏感，但可跨机（取决于实现）

10.3 两次 All-to-All：MoE 的通信主场

```
① Dispatch（发出去）
   每卡上的 token 按路由结果，发往「持有目标 Expert」的卡
   输入布局：按「来源卡」分组的 token
   输出布局：按「目标 Expert」分组的 token

② Expert 计算（本地）
   每卡对自己持有的 Expert 做 FFN

③ Combine（收回来）
   把 Expert 的输出按原路发回 token 的原卡，按门控权重加权求和
```

```
GPU0 ──[token→E2]──▶ GPU1
GPU0 ◀──[结果回传]── GPU1
GPU1 ──[token→E0]──▶ GPU0
      ...每个 token 都可能跨卡...
```

项	说明
通信量	正比于 `tokens × K × h × dtype`（dispatch 与 combine 各一次）
为什么是瓶颈	每层都要做、量随 token 数线性增长、All-to-All 对网络拓扑不友好（全连接模式）
优化	分组 All-to-All（按目标卡分桶）、计算通信重叠、专用通信库（如 DeepEP）、FP8 传输

> 一个反直觉的比较：DDP 的 AllReduce 也是每步一次，但它只发生一次、且是**规约**（数据量不随卡数涨）；MoE 的 All-to-All 是**每层两次**、且是**全交换**。所以 MoE 训练的网络压力通常显著高于同等规模的稠密模型【推断】。

10.4 负载均衡：MoE 的头号工程问题

Router 是学出来的，它天然倾向于「少数 Expert 被过度选择」：

```
理想：                    实际（路由倾斜）：
E0 ███████                E0 ████████████████████  ← 过载，OOM/排队
E1 ███████                E1 ████
E2 ███████                E2 ██
E3 ███████                E3 ██
```

后果：部分 GPU 过载（甚至 OOM）、部分空闲 → 整体吞吐被最慢的那张卡拖死（与 PP 的 Bubble 同构的问题）。

手段	做法	优缺点
Auxiliary Loss	加一项「让路由分布接近均匀」的辅助损失	简单有效；但会**干扰主任务**（aux loss 权重难调，且可能损害模型质量）
Capacity + Drop	限制每 Expert 容量，超了丢 token	保证不 OOM；丢 token 损失信息
Aux-loss-free（DeepSeek-V3）	用**每个 Expert 的偏置项**动态调整路由分数，不用 aux loss	不干扰主目标；实现更复杂，需要维护 bias 更新规则
Expert 复制 / 重排	把热门 Expert 复制到多卡	增加显存；需要一致性处理

> DeepSeek-V3 的 aux-loss-free 是近年最重要的工程进展之一：它在路由 logit 上加一个**与 expert 负载相关的 bias**，训练中按负载动态 $\pm\gamma$ 调整，从而在不引入 auxiliary loss 的前提下实现均衡。这个设计的价值在于——aux loss 本质是「用主任务的梯度去换均衡」，而 bias 调整是「用外部控制量去换均衡」，**不污染主任务梯度**。

10.5 EP × DP × TP × PP 的组合

组合	怎么切	注意
EP × DP	EP 组内的卡共享同一批 Expert；DP 组之间是数据副本	**EP 组必须与 DP 组正交**：同一 EP 组内的卡不能同时是 DP 副本
EP × TP	单个 Expert 内部再做张量并行	只对「超大 Expert」有意义；否则 TP 的通信开销不划算
EP × PP	MoE 层与稠密层在流水线中的分配	MoE 层计算量不均 → PP 的 Stage 切分要按 FLOPs 而非层数

```
一个 4D 布局的示意（TP=2, EP=2, PP=2, DP=2 → world_size=16）：

rank 0..15 按（tp, ep, pp, dp）四维坐标编排
同一 TP 组：{0,1}       ← 机内 NVLink
同一 EP 组：{0,2}       ← All-to-All，尽量机内
同一 PP 组：{0,4}       ← 跨机 P2P
同一 DP 组：{0,8}       ← 跨机 AllReduce（梯度）
```

> 【推断】rank 编排的一般原则：**通信最频繁的维度放在物理最相邻的位置**（同机、同 NVLink 域、同 leaf switch）。EP 的 All-to-All 是「全连接」模式，对 leaf-spine 拓扑最不友好，应尽量约束在一个 NVLink 域或一个 pod 内。

10.6 本章小结与自检

1. MoE 用「显存换容量」：参数随 E 涨、计算只随 K 涨。
2. EP 切的是「Expert 放到哪张卡」，通信是每层两次 All-to-All（dispatch + combine）。
3. 负载均衡是头号问题：aux loss 有效但污染主任务；aux-loss-free 用 bias 调整绕开它。
4. EP 与 DP 必须正交编排；EP 的 All-to-All 对网络拓扑最敏感。
5. Capacity Factor 与 token drop 是「精度 vs 显存/吞吐」的旋钮。

自检题：

1. E=64、K=2 的 MoE，参数量是稠密模型的多少倍？单 token 计算量是几倍？
2. 为什么 MoE 的显存不随 K 减小而减小？
3. 画出 dispatch → 计算 → combine 的三步数据流，标出两次 All-to-All 的位置。
4. aux loss 与 aux-loss-free 的本质区别是什么？各有什么代价？

第 11 章 3D 并行与混合并行：把五个维度装进一个集群

> 对应 AIInfraGuide 模块三 第 11 章（`第11章-3d并行与混合并行策略` 扉页）。

11.1 为什么必须组合

单一策略	天花板
DP	不省参数/状态显存；512+ 卡受环延迟限制

单一策略	天花板
TP	出不了机（每层都通信）；并行度受 head 数/层宽限制
PP	有 Bubble；层数必须够分（P ⩽ L）
ZeRO-3	通信 +50%；单层太大时仍装不下
EP	只对 MoE 有意义

维度正交性：不同并行切的是不同东西，可以叠加。约束只有一个：

```
world_size = TP × PP × DP × EP × CP
```

（严格说 EP 常与 DP 共享一部分卡，实际写法因框架而异；Megatron 里通常是 `world = TP × PP × CP × DP`，EP 从 DP 里再切。**看框架文档，别硬套公式**。）

```
                    ┌──────────── 5D 拼装 ────────────┐
DP  ── 复制数据与模型副本（跨机）                      │
 └─ ZeRO/FSDP ── 在 DP 组内切分训练状态                 │
      └─ PP  ── 把层切成 Stage（跨机）                  │
           └─ TP  ── 切每层的矩阵（机内）               │
                └─ CP ── 切序列（Attention）            │
                     └─ EP ── 切 Expert（MoE 层）       │
```

11.2 通信域映射：本章最重要的一节

原则只有一句：**按通信频率/带宽需求排序，映射到带宽从高到低的硬件**。

维度	通信时机	通信量级	带宽需求	映射到
TP	每层 2 次 AllReduce	大（4bsh/层）	**最高**（延迟敏感）	机内 NVLink（⩽8 卡）
CP	每层 Attention 内	中大（KV 交换）	高	机内 / 同 pod
EP	每层 2 次 All-to-All	大（全交换）	高（拓扑敏感）	机内 / 同 pod
PP	每个 micro-batch 边界	小（b·s·h）	中	可跨机 IB
DP	每步 1 次 AllReduce	大但次数少	最低（可重叠）	跨机 IB / 以太网

```
通信量从大到小：TP > CP ≈ EP > PP > DP
硬件带宽从高到低：NVLink > NVSwitch 域 > IB > 以太网
          ↓
      按序对齐，就是并行维度的「由内到外」布局
```

> 这张表是「为什么 TP 在内、DP 在外」的完整答案。**如果你只记一句话**：把最贵的通信放在最快的线上。

11.3 rank 编排：从一维编号到多维坐标

`world_size` 个 rank 是一维的，但并行是多维的。需要一次映射：

```
rank → (dp_rank, pp_rank, tp_rank)      常见顺序（Megatron 风格）：
                                          rank = dp·(PP·TP) + pp·TP + tp
```

8 卡、TP=2、DP=4 的例子：

```
TP 组（组内 AllReduce）：  {0,1} {2,3} {4,5} {6,7}      ← 相邻 rank 成组（同机）
DP 组（组内 AllReduce）：  {0,2,4,6} {1,3,5,7}          ← 跨 TP 组、同 TP 位置
```

```
# 构造子通信组（第 12 章会给完整脚本）
tp_size, dp_size, rank = 2, 4, dist.get_rank()
tp_groups = [list(range(i, i + tp_size)) for i in range(0, tp_size * dp_size, tp_size)]
tp_group = None
for ranks in tp_groups:
    g = dist.new_group(ranks=ranks)     # 所有进程都要调用！
    if rank in ranks:
        tp_group = g
# 之后: dist.all_reduce(t, group=tp_group)
```

编排原则	理由
TP 组用相邻 rank	相邻 rank 通常落在同一台机器（torchrun 按 local_rank 顺序分配），天然共享 NVLink
PP 组跨节点	PP 通信量小，可以承受跨机
DP 组最外层	DP 通信可被计算重叠，对带宽最不敏感
`new_group` 必须全员调用	否则死锁（每个进程都要参与组建，只是不一定加入）

11.4 拓扑设计实战：64 卡（8 节点 × 8 GPU）

给定模型：80 层、h=8192、单层参数约 1.6B【量级】，总参数约 130B。要求单卡装得下，且吞吐尽量高。

方案	TP	PP	DP	每卡参数显存（粗算）	PP Bubble（M=16）	评价
A	8	4	2	130B/32 ≈ 4B 参数 ≈ 8 GB	(4−1)/(16+3) ≈ 15.8%	TP 吃满整机，PP 跨机，**推荐**
B	4	8	2	同上（TP×PP=32）	(8−1)/(16+7) ≈ 30.4%	Bubble 翻倍，TP 只用半机

方案	TP	PP	DP	每卡参数显存（粗算）	PP Bubble（M=16）	评价
C	8	8	1	130B/64 ≈ 2B ≈ 4 GB	(8−1)/(16+7) ≈ 30.4%	显存最省但 Bubble 大、无 DP 冗余

取舍逻辑（原文给的方法论）：

```
1）先满足「装得下」：TP × PP ≥ 总参数 × 16 / 单卡容量
2）再压 Bubble：PP 尽量小、M 尽量大（受显存限制）
3）再看网络：TP ≤ 单机卡数（8）；PP 可跨机
4）最后用 DP 填满剩余卡数（DP 最便宜）
```

> ⚠️ 一个常被忽略的约束：**TP 度不能超过单层能切的最小粒度**。若某层只有 8 个 head，TP=16 就需要复制 KV（第 6 章），收益递减。同理 PP 度不能超过层数（P ⩽ L）。

11.5 显存与吞吐的联合估算

给定 (TP, PP, DP)，每卡显存：

```
M_card ≈ 16Ψ/(TP·PP·DP)                    ← 训练状态（若开 ZeRO-3 再除 DP 内的分片度）
       + 激活(b, s, h, L/PP) × (1/TP 或 1/CP) ← 只算本 Stage 的层
       + 通信缓冲（bucket、AllGather 峰值）
       × 1.2~1.5（碎片与安全系数）
```

端到端吞吐：

```
T_step ≈ max( T_compute , T_TP_comm , T_EP_comm ) + T_PP_bubble + T_DP_comm_exposed
```

项	怎么估	怎么优化
`T_compute`	`6·Ψ_local·tokens / (FLOPS × MFU)`	提 MFU（融合算子、FlashAttention）
`T_TP_comm`	每层 4bsh / 带宽 × 层数	提高 TP 只在机内；用 SP 减少激活
`T_PP_bubble`	`(P−1)/(M+P−1) × T_step`	增大 M、Interleaved、减小 P
`T_DP_comm`	`2Ψ_local / 带宽`（与计算重叠后取剩余）	bucket 调优、`no_sync` 累积、HYBRID_SHARD

调参方法论（原文的核心建议）：先满足「装得下」，再优化「跑得快」。顺序错了会白折腾——先追 MFU 再发现 OOM，等于重来。

11.6 一张「选型决策表」收尾

集群规模	模型规模	推荐组合	理由
8 卡单机	≤7B	DDP 或 ZeRO-2	简单，通信最省
8 卡单机	7B~13B	ZeRO-3 / FSDP FULL_SHARD	16Ψ/N 刚好装下
8 卡单机	30B+	TP=8 + ZeRO-1	单层也大，必须切层内
32 卡（4×8）	70B	TP=8, PP=2, DP=2	TP 吃满整机，PP 跨机
64 卡（8×8）	130B	TP=8, PP=4, DP=2	见 11.4 方案 A
512 卡+	400B+	TP=8, PP=8~16, DP=…, 视情况 +CP/EP	全维度组合，看网络

11.7 本章小结与自检

1. `world_size = TP × PP × DP × EP × CP`；维度正交，可以叠加。
2. 通信量排序 TP > CP ≈ EP > PP > DP，硬件带宽排序 NVLink > IB > 以太网，**按序对齐**。
3. rank 编排：TP 用相邻 rank（同机）、PP 跨机、DP 最外层；`new_group` 必须全员调用。
4. 设计顺序：先装得下 → 再压 Bubble → 再看网络 → 最后用 DP 填满。
5. 吞吐 = max(计算, 各维度通信) + Bubble + 未重叠的 DP 通信。

自检题：

1. 96 卡（12 节点 × 8）、模型 175B、单卡 80GB，给出一个可行的 (TP, PP, DP) 方案并说明理由。
2. 为什么 TP 组要用「相邻 rank」而不是「每隔 8 个取一个」？
3. `world_size = TP×PP×DP` 与「EP 从 DP 里再切」这两种写法有什么区别？影响什么？
4. 你的 3D 配置下 MFU 只有 25%，列出三个最可能的瓶颈和验证方法。

第 12 章 实战：环境、torchrun 与排障

> 对应 AIInfraGuide 模块三 第 1 章后半（`12-环境搭建与分布式启动` 正文）。

12.1 三种规模与带宽分层

规模	典型配置	互联	典型场景
单机单卡	1 GPU	—	调试、小模型原型

规模	典型配置	互联	典型场景
单机多卡	8 GPU / 节点	NVLink / NVSwitch	中等模型、TP
多机多卡	N 节点 × 8 GPU	InfiniBand / RoCE	百亿~千亿参数

> 关键认知是**带宽分层**：机内 NVLink 数百 GB/s，跨机 IB 大约只有机内的几分之一到十分之一。比喻：一栋楼里的同事可以走过去当面讨论，跨城市团队只能开视频会——**能当面说的事尽量别开远程会**。

12.2 一卡一进程与三个编号

为什么不用单进程多线程管 8 张卡？两个原因：

1. **绕开 GIL**：Python 全局解释器锁让同进程多线程无法真正并行执行字节码，调度逻辑会被串行化。
2. **独占 CUDA 上下文**：每进程绑一块卡，显存空间干净、互不干扰、崩了只影响一个进程。

```
Node0 (node_rank=0)                Node1 (node_rank=1)
├ 进程 rank=0, local_rank=0 → GPU0    ├ 进程 rank=2, local_rank=0 → GPU0
└ 进程 rank=1, local_rank=1 → GPU1    └ 进程 rank=3, local_rank=1 → GPU1
```

概念	含义	用途
`rank`	全局唯一编号，0 ~ world_size−1	`rank == 0` 当主进程（打日志、存 ckpt）
`local_rank`	节点内编号，每台机器都从 0 开始	**`torch.cuda.set_device(local_rank)`**
`world_size`	总进程数 = nnodes × nproc_per_node	初始化进程组

```
rank = node_rank × G + local_rank            (G = 每节点进程数)
world_size = nnodes × G
```

> ⚠️ 初学者最容易犯的错：用 `rank` 去 `set_device`。多机时 `rank=2` 在第二台机器上根本没有「第 2 块卡」。**决定 GPU 绑定的永远是 local_rank。**

12.3 torchrun：一条命令拉起一堆进程

老接口 `python -m torch.distributed.launch` 已废弃（靠 `--local_rank` 命令行参数传递，无弹性）。现在统一用 `torchrun`（等价于 `python -m torch.distributed.run`）。

12.3.1 注入的六个环境变量

变量	含义	典型用法
`RANK`	全局进程编号	`init_process_group` 的 rank
`LOCAL_RANK`	节点内编号	`torch.cuda.set_device(local_rank)`
`WORLD_SIZE`	总进程数	初始化进程组
`LOCAL_WORLD_SIZE`	本节点进程数	估算本机资源
`MASTER_ADDR`	主节点 IP / 主机名	会合地址
`MASTER_PORT`	会合端口（需空闲）	会合端口

```
import os
rank = int(os.environ["RANK"])
local_rank = int(os.environ["LOCAL_RANK"])
world_size = int(os.environ["WORLD_SIZE"])
# MASTER_ADDR / MASTER_PORT 通常不用手动读：
# env:// 初始化方式下 init_process_group 会自己从环境变量取
```

12.3.2 单机多卡

```
torchrun --nproc_per_node=8 train.py --epochs 10 --lr 1e-4
# 背后：起 8 个进程 → 写入 RANK=0..7 / LOCAL_RANK=0..7 / WORLD_SIZE=8
#       → 设 MASTER_ADDR=127.0.0.1 + 空闲端口 → 本机会合
```

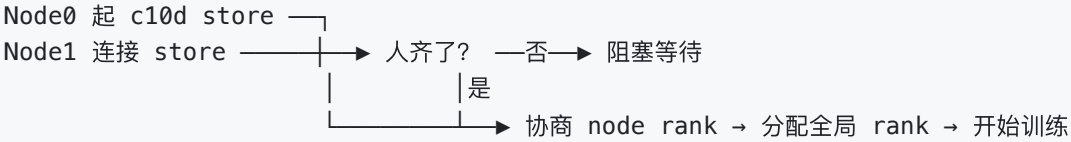
> 单机下 `RANK == LOCAL_RANK`，所以用哪个「看起来都对」。**一上多机这个等式就破了**——这就是反复强调 local_rank 的原因。

12.3.3 多机：c10d 弹性会合

```
# Node0（rdzv 集合点，IP 192.168.1.1）与 Node1 执行完全相同的命令：
torchrun \
    --nnodes=2 \
    --nproc_per_node=8 \
    --rdzv_id=my_job \
    --rdzv_backend=c10d \
    --rdzv_endpoint=192.168.1.1:29400 \
    train.py
```

参数	含义
`--nnodes=2`	总节点数；`WORLD_SIZE = 2 × 8 = 16`

参数	含义
`--rdzv_id`	任务唯一标识，所有节点必须相同
`--rdzv_backend=c10d`	PyTorch 内置会合实现，集合点起一个 TCP store
`--rdzv_endpoint`	集合点地址，所有节点填同一个



> ⚠️ 用 c10d 动态会合时，**节点编号由会合过程自动协商**，各节点命令完全一致，不需要 `--node_rank`。`--node_rank` / `--master_addr` / `--master_port` 是早期静态启动的写法，**别和弹性写法混用**。

弹性能力（`--nnodes=2:4` 允许 2~4 节点动态伸缩）配 checkpoint 续训才能真用起来——这是第 14 章（原文的第 14 章）容错主题的内容。

12.4 子通信组：为 3D 并行铺路

```
tp_size, dp_size = 2, 4
rank = dist.get_rank()

tp_groups = [list(range(i, i + tp_size)) for i in range(0, tp_size * dp_size, tp_size)]
dp_groups = [[i + j * tp_size for j in range(dp_size)] for i in range(tp_size)]

tp_group = dp_group = None
for ranks in tp_groups:                  # 所有进程都要遍历所有组
    g = dist.new_group(ranks=ranks)
    if rank in ranks:
        tp_group = g
for ranks in dp_groups:
    g = dist.new_group(ranks=ranks)
    if rank in ranks:
        dp_group = g

# 使用时显式指定通信域
# dist.all_reduce(t, group=tp_group)
```

> 📌 `new_group` 必须被**所有**进程调用（即使不属于该组），否则死锁。每个进程最终只持有「自己所在那个子组」的句柄。

12.5 排障流程

```
多卡 hang / 慢 / 报错
   │
   ├─ 第一步：export NCCL_DEBUG=INFO   ← 永远是这一招
   │
   ├─ 卡在 init_process_group?
   │     ├─ 检查 MASTER_ADDR / 端口 / 防火墙
   │     ├─ 检查是否所有节点都起来了（少一个就永久等）
   │     └─ 检查 NCCL_SOCKET_IFNAME（网卡选错会 hang 在初始化）
   │
   ├─ 机间没用上 IB?
   │     ├─ 设 NCCL_IB_HCA
   │     └─ 确认 IB 驱动 / 容器内设备透传
   │
   ├─ 机内没走 NVLink?
   │     ├─ 用 nvidia-smi topo -m 看拓扑
   │     └─ 试 NCCL_P2P_DISABLE=1 二分排查
   │
   └─ 容器内失败?
         ├─ 检查 /dev/shm 大小（默认 64MB 太小）
         └─ 试 NCCL_SHM_DISABLE=1，或用 --shm-size 调大
```

12.6 首次跑多卡的五个坑

坑	现象	解法
端口占用	`Address already in use`	换 `MASTER_PORT`；`pkill -f train.py` 清残留
卡在 init	无输出、永久 hang	`NCCL_DEBUG=INFO` 重跑；查 MASTER_ADDR、网卡、节点是否齐
全挤 0 号卡	0 卡 OOM，其他卡很空	忘了 `set_device(local_rank)`；检查每个 `.to(device)`
loss 不一致/不收敛	各卡 loss 不同	忘了 `DistributedSampler` / `set_epoch`；或模型有未使用参数
NCCL/驱动不匹配	`NCCL error` / `unhandled cuda error`	对齐 PyTorch 自带 NCCL 与驱动；容器检查 `/dev/shm`

12.7 本章小结

1. 三档规模 + 带宽分层，是策略布局的物理基础。
2. 一卡一进程（绕 GIL、独占 CUDA 上下文）；rank 全局、local_rank 绑卡、world_size 总数。

3. torchrun 注入六个环境变量；单机 `--nproc_per_node`，多机 c10d 弹性会合（各机命令相同）。
4. `new_group` 派生通信域，全员调用否则死锁——这是 3D 并行的地基。
5. 排障永远是：`NCCL_DEBUG=INFO` → 看链路 → 二分开关 → 恢复默认。

第 13 章 与已有笔记的衔接：把分布式训练接回你学过的课

分布式训练不是一门孤立的工程技能，它几乎是「系统课 + 网络课 + 体系结构课」的一次联合考试。下面这张表把本模块的每个概念映射回你已有的笔记。

本模块的概念	对应课程 / 笔记	接的是哪个知识点	怎么用
AllReduce / Ring 算法	CS144 计算机网络	带宽、延迟、拥塞、流控	把「通信量 / 带宽 + α」当成一次 RTT 估算；理解为什么「带宽最优」与「延迟最优」是两件事
NCCL 拓扑与路径选择	CS144 + CSAPP	分层、路由、NUMA、PCIe	看懂 `NCCL_DEBUG=INFO` 里的 `via NVLink` / `via PCIe` / `NET/IB`
TCP store / Rendezvous	CS425 分布式系统	成员管理、故障检测、协调服务	`init_process_group` 就是一次分布式成员协商；理解了它就理解「为什么少一个进程会 hang」
同步 SGD 的等价性	CS425	一致性模型、同步 vs 异步复制	数据并行的「严格等价」只在同步 + 等大 batch 下成立，正是分布式一致性的老问题
容错与弹性训练	CS425	副本、检查点、崩溃恢复	`torchrun --nnodes=2:4` + checkpoint 续训 = 弹性副本管理
Warp / SM / 占用率	CS149 并行计算	并行执行模型、负载均衡、Amdahl	PP 的 Bubble、TP 的通信开销，本质都是「并行度上去了，串行部分没减少」
内存层次	CSAPP	寄存器/L1/L2/DRAM/NVMe 的容量–延迟–带宽三角	显存分层（HBM / L2 / SMEM / 寄存器）与 Offload（DRAM / NVMe）是同一套「层次」思想
虚拟内存与分页	CSAPP	页表、缺页、预取	ZeRO-3 的 AllGather-on-demand 与 Offload 的 prefetch，就是「按需调页 + 预取」
并行算法复杂度	CS149	work / span、通信复杂度	`2(N-1)/N·Ψ` 就是通信复杂度的具体化；`span` 对应 Ring 的 `2(N-1)` 步延迟
数值精度	任意数值课 / CSAPP 浮点章	IEEE 754、舍入、非正规数	为什么 master weights 必须 FP32（大数吃小数）、为什么 FP16 要 loss scaling

三条「迁移过来就能用」的判断力：

1. **「带宽最优 ≠ 延迟最优」**（CS144）：Ring 是带宽最优但步数 O(N)，Tree 是延迟最优但带宽略差。选哪个取决于你的消息大小——小消息看 α，大消息看带宽。
2. **「并行加速比受串行部分限制」**（CS149 / Amdahl）：PP 的 Bubble、DDP 的梯度同步、TP 的每层 AllReduce，都是「串行部分」。加卡到一定程度，这些部分占比上升，加速比就饱和了。
3. **「层次结构决定优化方向」**（CSAPP）：显存的层次是 HBM → L2 → SMEM → 寄存器，训练优化的层次是 参数/梯度/优化器/激活 → 切分/降精度/重计算/卸载。**先定位你在哪一层卡住，再选手段。**

第 14 章 复习清单与自测题

14.1 分章复习清单

章	必须能独立完成的事	过关标志
1	手算任意参数量模型的 16Ψ 与 7B/70B 的静态显存	30 秒内报出数并判断单卡能否装下
1	用 `6ΨD` 估算单卡训练时间并推算 N 卡后的量级	能说清「26.6 年 → 9 天」的每一步
1	画出五大策略的「切什么 × 怎么通信 × 解决什么」表	不看笔记能默画
2	默画 7 个集合原语的输入输出语义图	每个都能举一个并行策略作为用途
2	手推 Ring 的 `2(N−1)/N·Ψ` 并解释「与 N 无关」	能说出 `2(N−1)` 步的延迟代价
2	用 `T ≈ α + 数据量/B` 估算一次 AllReduce	会正确代入机内 / 机间带宽
3	说清 SGD → Momentum → Adam → AdamW 各多存几份状态	能解释 AdamW 解耦 weight decay 的意义
3	用代码统计 PyTorch 优化器 state 的元素数	验证 Adam 存 2 份、SGD 存 0 份
4	推导「N 卡平均梯度 = 单卡全局 batch 梯度」	能解释为什么不能求和
4	写一个能跑通的 DDP 脚本（含 sampler、set_epoch、rank0 守卫）	`torchrun` 起来 loss 正常下降
5	默写 ZeRO-1/2/3 的每卡显存公式	能算 7B@8 卡的三档数值
5	说清 FSDP 一个 unit 的 AllGather/ReduceScatter 流程	能解释 3Ψ 的每一份 Ψ 从哪来
6	画出 TP=4 的 Block 切分图并标出通信位置	能解释「Column→Row 中间不用通信」
6	说清 SP 如何把 AllReduce 拆成 RS + AG	能解释「通信量不变、激活降 N 倍」
7	默写三种调度的 Bubble 公式	能解释 1F1B 改善的是显存不是 Bubble
8	说清 BF16 混合精度流程与 FP32 主权重的必要性	能解释 FP16 为什么必须 loss scaling
8	写出带 `no_sync()` 的梯度累积循环	能说明 loss 要除以 K
9	说清 Ring / Ulysses / Megatron CP 的切分维度差异	能按 Head 数选方案

章	必须能独立完成的事	过关标志
10	画出 MoE 的 dispatch→计算→combine	能解释为什么负载均衡是头号问题
11	给定集群与模型，设计一套 (TP, PP, DP) 并说明理由	能算 Bubble 与每卡显存
12	写出单机与多机 torchrun 命令	能说出六个环境变量的含义

14.2 自测题（20 题，答案在后）

A 组：显存账本

1. 13B 模型，BF16 + AdamW，静态显存多少？需要几张 80GB 卡「刚好」装下（不含激活）？
2. 若把优化器从 AdamW 换成 SGD（无 momentum），同一个 13B 模型的静态显存变成多少？降幅百分比？
3. 一个模型参数 2Ψ 是 40 GB，训练时单卡却 OOM 在 80GB 卡上。列出至少四种可能吃掉显存的项，并给出对应手段。
4. 为什么说「7B 模型推理能单卡跑，训练不能单卡训」？分别给数字。

B 组：通信

1. 8 卡、AllReduce 一个 14 GB 的梯度，机内 300 GB/s、机间 50 GB/s，各需多久？如果一次迭代计算需要 0.3 s，哪种情况通信会被暴露？
2. 手推 Ring AllReduce 每卡通信量，并说明为什么它与卡数无关。
3. 为什么 FSDP 是 3Ψ 而 DDP 是 2Ψ？多出来的那份 Ψ 能不能省掉？
4. `NCCL_DEBUG=INFO` 显示机间走的是 socket 而不是 IB。写出至少三条排查动作。

C 组：并行策略

1. 为什么 TP 必须放机内，而 PP 可以跨机？用通信量数字回答。
2. ZeRO-2 的通信量「看起来」和 DDP 一样，它到底省了什么、补了什么？
3. 1F1B 和 GPipe 的 Bubble 公式相同，为什么大家选 1F1B？
4. P=8、M=16 时 1F1B 的 Bubble 是多少？要压到 10% 以下需要多大的 M？
5. SP 之后总通信量为什么没变？收益在哪？
6. 128K 序列训练，为什么 PP 救不了？该上什么？
7. Ulysses 在 MQA（1 个 KV Head）上能开吗？为什么？替代方案是什么？
8. E=64、K=2 的 MoE，参数量与单 token 计算量分别是稠密模型的多少倍？

D 组：工程

1. 写出单机 8 卡与 2 机 16 卡的 torchrun 命令。
2. `rank` 与 `local_rank` 的区别？用错会怎样？
3. `new_group` 为什么必须所有进程都调用？

4. 你的 3D 配置 MFU 只有 25%，列出三个最可能的瓶颈和各自的验证方法。

14.3 答案

1. `16 × 13×10⁹ = 2.08×10¹¹ B ≈ 208 GB`。80GB 卡：`208/80 = 2.6` → **至少 3 张**（实际还要留激活和碎片，通常 4 张起步）。

2. SGD 无 momentum：`2Ψ(参数) + 2Ψ(梯度) + 4Ψ(FP32 主权重) = 8Ψ`。`8 × 13B = 104 GB`。降幅 `(208−104)/208 = 50%`。省下的正是 m 和 v 两份各 4Ψ。

3. 可能项：(a) 优化器状态 12Ψ（若 2Ψ=40GB 则 12Ψ=240GB，直接爆）→ ZeRO-1/2；(b) 激活值（长序列/大 batch）→ 减 batch、CKPT、SP/CP；(c) 通信 bucket / AllGather 峰值缓冲 → 调 `bucket_cap_mb`、`limit_all_gathers`；(d) 显存碎片 → `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`；(e) 未释放的中间张量（忘了 `del` / 保留了 loss 计算图）→ 检查代码。

4. 推理只需 BF16 权重 `2Ψ = 14 GB` + KV Cache，单卡 80GB 轻松。训练需要 `16Ψ = 112 GB`（2 参数 + 2 梯度 + 12 优化器）+ 激活，超 80GB。**差的那 98 GB 全是「训练才有的状态」。**

5. 机内：`2×14/300 ≈ 0.093 s`；机间：`2×14/50 ≈ 0.56 s`。计算 0.3 s 时：机内通信能被重叠掉（0.093 < 0.3）；机间 **0.56 > 0.3，通信完全暴露**，GPU 一半以上时间在等。这就是「TP 出不了机」的算术版。

6. 数据切 N 份，每份 Ψ/N；两阶段各 N−1 步，每步发一个分片：`(N−1)·Ψ/N × 2 = 2(N−1)/N·Ψ`。当 N 大时 `(N−1)/N → 1`，故趋于 `2Ψ`，**与 N 无关**。物理直觉：分片数随 N 增加，但每片大小同比例减小，两个效应抵消。

7. DDP：`AllReduce = RS(Ψ) + AG(Ψ) = 2Ψ`。FSDP：前向 AG(Ψ) + 反向 AG(Ψ) + 反向 RS(Ψ) = 3Ψ。多出的那份是**前向的 AllGather**——因为前向后参数被释放了，反向必须再拼一次。**理论上可以省**：如果显存够、不释放参数（退化成 ZeRO-2/`SHARD_GRAD_OP`），就不需要第二次 AG——这也正是 `SHARD_GRAD_OP` 通信更省的原因。

8. (a) `NCCL_IB_HCA` 指定正确的 HCA；(b) 检查 IB 驱动/`ibstat`、容器是否透传了 `/dev/infiniband`；(c) 检查 `NCCL_IB_DISABLE` 是否被误设为 1；(d) 检查 `NCCL_SOCKET_IFNAME` 是否指向了错误的网卡导致 fallback；(e) 看 NCCL 日志里是否有 `NET/IB` 初始化失败的告警。

9. TP 每层 2 次 AllReduce，单次张量 `b·s·h`（口径 `2bsh`），7B/32 层量级是 GB 级/步且**每层一次**，延迟极敏感 → 机内 NVLink。PP 只在 Stage 边界传 `b·s·h` 的激活，一次约 16 MB（b=1,s=2048,h=4096,BF16），量小 → 可跨机 IB。**差两个数量级。**

10. 省了：梯度不再每卡存完整 2Ψ，只存 `2Ψ/N`。补了：AllReduce 拆成 ReduceScatter（反向，得本人分片的聚合梯度）+ AllGather（更新后同步参数）。**总量仍是 2Ψ 量级，但峰值显存降下来了。**

11. 因为 1F1B 把**峰值激活**从 M 份降到 P 份（同一时刻最多 P 个 micro-batch 的激活存活）。同样的 Bubble、一半的显存 → 可以把 M 开得更大，从而**间接**把 Bubble 压得更小。GPipe 的 M 一大就爆显存，所以它的「M 越大越好」在实践中不可行。

12. `(8−1)/(16+8−1) = 7/23 ≈ 30.4%`。要 <10%：`7/(M+7) < 0.1 → M+7 > 70 → M ≥ 64`。也可以上 Interleaved（v=2 时等效 M 翻倍）。

13. 因为 SP 只是把 AllReduce **拆成** ReduceScatter + AllGather 两步做（进出非 TP 区各一步），总量不变。收益在**激活显存**：非 TP 区域从完整 `b·s·h` 降到 `b·s·h/N`，总激活接近线性缩放。

14. PP 切的是层，但**每一层内部的单条序列仍是完整的**，Attention 的 `O(s²)` 激活照样爆。必须沿序列切 Attention 计算本身 → 上 CP（Ring Attention / Ulysses / Megatron CP）。

15. 不能（或只能退化）。Ulysses 要求 **Head 数 ≥ CP 度**才能按 Head 切；MQA 只有 1 个 KV Head（Q head 可能多，但 KV 共享）。替代：Ring Attention（不依赖 Head 数），或对 Q head 切 + KV 复制的混合方案。

16. 参数量 ≈ **64 倍**（E=64 个 Expert）；单 token 计算量 ≈ **2 倍**（K=2，只过 2 个 Expert）。这就是 MoE「显存换容量」的定量表达。

17.

```
# 单机 8 卡
torchrun --nproc_per_node=8 train.py
# 2 机 16 卡（两台执行相同命令）
torchrun --nnodes=2 --nproc_per_node=8 \
  --rdzv_id=job1 --rdzv_backend=c10d \
  --rdzv_endpoint=192.168.1.1:29400 train.py
```

18. `rank` 是全局唯一编号（0~world_size−1），`local_rank` 是节点内编号（每台机器都从 0 开始）。用 `rank` 去 `set_device`：多机时第二台机器的进程会尝试绑定「第 rank 块卡」，越界或绑错卡 → 报错或所有进程挤到同一张卡上 OOM。

19. `new_group` 是一次**集合操作**：所有进程都必须参与通信组的创建（底层要建立 communicator），只是不一定加入。若某些进程跳过调用，其余进程会永久等待 → 死锁。

20. 三个候选：(a) **PP Bubble 过大**——验证：算 `(P−1)/(M+P−1)`，看是否 >15%；手段：增大 M、Interleaved、减小 P。(b) **TP 通信暴露**——验证：`NCCL_DEBUG=INFO` 确认走 NVLink；用 nsys 看 AllReduce 是否与计算重叠；手段：TP 限制在机内、用 SP 减激活。(c) **DP 通信未重叠**——验证：对比开/关 `gradient_as_bucket_view`、调 `bucket_cap_mb`；手段：梯度累积 + `no_sync`、HYBRID_SHARD。(d) 还有可能是**数据加载瓶颈**——验证：看 GPU 利用率是否周期性掉到 0；手段：加 `num_workers`、预取。

第 15 章 参考来源

15.1 一手来源（AIInfraGuide 模块三，访问日期 2026-11）

页面前缀：`https://caomaolufei.github.io/AIInfraGuide/distributed/模块三-分布式训练/`

#	页面	类型	本笔记用它来支撑
1	模块首页 `…/distributed/`	索引	17 篇的目录与顺序

#	页面	类型	本笔记用它来支撑
2	`第1章-分布式训练总论`	扉页	显存四大组成、五大策略、决策树、学习目标
3	`11-分布式训练总论`	**长篇正文**	16Ψ 账本、6ΨD、112/1120/6480 GB、带宽表、决策树
4	`12-环境搭建与分布式启动`	**长篇正文**	三档规模、进程模型、rank/local_rank、torchrun、NCCL 环境变量、DDP 模板、五个坑
5	`第2章-集合通信原语`	扉页	七原语语义、Ring 推导、α-β 模型、动手实验
6	`21-集合通信原语详解`	**长篇正文**	原语图示、死锁、Ring 推导、Tree/Double Binary Tree、NCCL_DEBUG
7	`第3章-优化器`	扉页	优化器演进、12Ψ、FP32 主权重、LARS/LAMB
8	`31-优化器原理与显存开销分析`	**长篇正文**	12/8/4 B 表、75% 占比、BF16 尾数论证、state 统计代码
9	`第4章-数据并行`	扉页	DP/DDP/FSDP 三代框架
10	`41-数据并行详解`	**长篇正文**	数学等价推导、DP 三缺陷、Bucket、2Ψ/3Ψ、四种 sharding、选型树
11	`42-pytorch-数据并行从原理到实战`	**长篇正文**	DDP 实战、FSDP 完整代码、DCP 保存、FSDP2、torchrun 实战
12	`第5章-zero系列`	扉页	ZeRO-1/2/3 公式、Offload/Infinity、ZeRO vs FSDP
13	`第6章-张量并行与序列并行`	扉页	Column/Row Parallel、TP 切分图、SP、GQA
14	`第7章-流水线并行`	扉页	朴素/GPipe/1F1B/Interleaved 的 Bubble 与显存
15	`第8章-混合精度与显存优化`	扉页	四种精度表、Loss Scaling、梯度累积、CKPT
16	`第9章-长序列训练与上下文并行`	扉页	$O(s^2)$、Ring/Ulysses/Megatron CP 对比
17	`第10章-moe并行`	扉页	Router、EP、两次 All-to-All、负载均衡
18	`第11章-3d并行与混合并行策略`	扉页	维度正交、通信域映射、rank 编排、64 卡实例

> 说明：第 2～18 行共 17 个页面 URL（与「模块三 17 篇」一致），其中 6 篇为长篇正文，11 篇为章节扉页（含完整章节结构、公式表与选型表）。本笔记的**章节骨架与所有 16Ψ/12Ψ/2Ψ/3Ψ/Bubble 公式**均来自这些页面；标注「补充」的内容来自下列来源。

15.2 补充来源（论文与官方文档）

主题	来源
ZeRO / 16Ψ / 三阶段公式	Rajbhandari et al., *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*, SC 2020
ZeRO-Offload / Infinity	Ren et al., *ZeRO-Offload* (ATC 2021); Rajbhandari et al., *ZeRO-Infinity* (SC 2021)
张量并行 / 序列并行	Shoeybi et al., *Megatron-LM*, 2019; Korthikanti et al., *Reducing Activation Recomputation in Large Transformer Models*, 2022
流水线并行	Huang et al., *GPipe*, NeurIPS 2019; Narayanan et al., *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*, SC 2021
数据并行 / FSDP	PyTorch 文档：*DistributedDataParallel*、*FullyShardedDataParallel*、`torch.distributed.checkpoint`、*torchrun (Elastic Launch)*
集合通信	NVIDIA *NCCL* 官方文档与 *nccl-tests*；Gibiansky, *Bringing HPC Techniques to Deep Learning*（Ring AllReduce 解析）
混合精度	Micikevicius et al., *Mixed Precision Training*, ICLR 2018; NVIDIA *Transformer Engine* 文档
FlashAttention	Dao et al., *FlashAttention* (NeurIPS 2022); *FlashAttention-2* (2023)
长序列 / CP	Liu et al., *Ring Attention with Blockwise Transformers for Near-Infinite Context*, 2023; Jacobs et al., *DeepSpeed Ulysses*, 2023
MoE	Shazeer et al., *Outrageously Large Neural Networks* (ICLR 2017); Fedus et al., *Switch Transformers*, 2021; DeepSeek-AI, *DeepSeek-V3 Technical Report*, 2024
显存与算力口径	NVIDIA A100/H100/B200 架构白皮书与产品规格页（注意区分稠密/稀疏口径）

15.3 阅读顺序建议

```
第一遍（建立地图）：第 1 章 → 第 4 章 → 第 5 章 → 第 11 章
第二遍（补机制）  ：第 2 章 → 第 3 章 → 第 6 章 → 第 7 章
第三遍（补前沿）  ：第 8 章 → 第 9 章 → 第 10 章
动手              ：第 12 章 + 配套《项目上手速查》
巩固              ：第 13 章（接回旧课）→ 第 14 章（自测）
```

最后一句提醒：本模块的所有系数（16、12、2、3、33%）都绑定「BF16 + Adam + 稠密 Transformer」这一个具体配置。换配置就要重算——**会推导比会背更重要**，这也是原文反复强调「手算一遍」的真正原因。